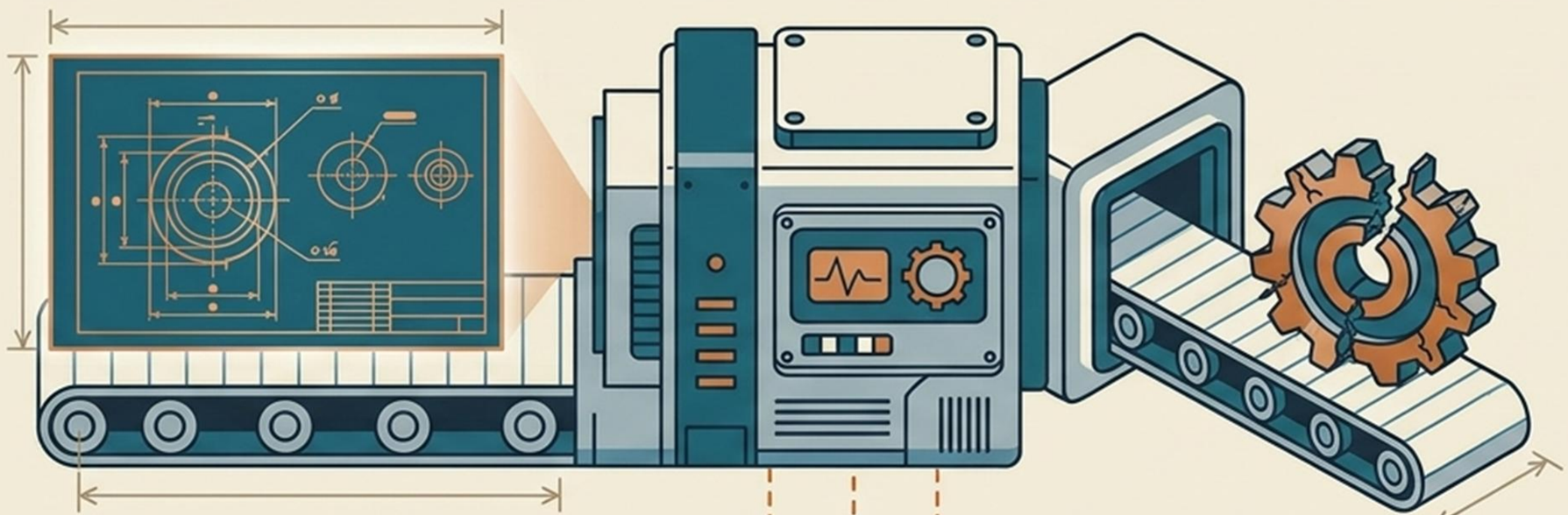
Desarrollo Ágil de Software

¿Por qué ágil?
Fundamentos y contexto

Antes de arrancar...



¿Alguna vez terminaron de desarrollar algo y el cliente dijo “esto no era lo que quería”?

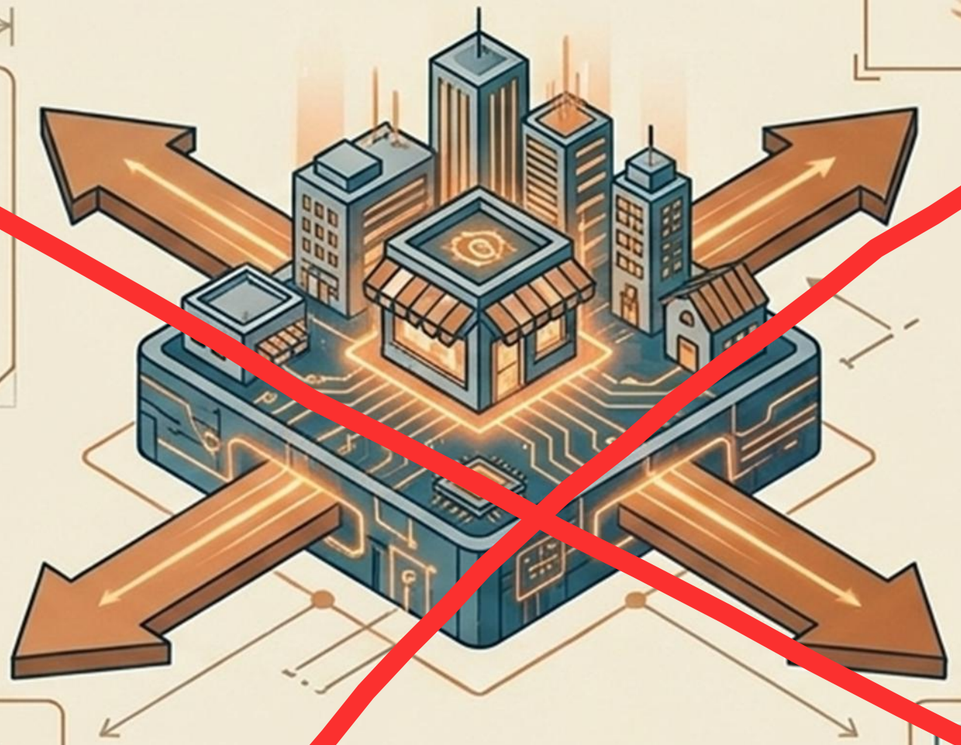
¿Conocen algún proyecto de software que haya llegado tarde o se haya cancelado?

¿Qué pasa cuando el cliente cambia los requerimientos a mitad del desarrollo?

El entorno en el que nace el software actual

Las organizaciones prefieren negociar perfección inicial a cambio de llegar a tiempo al mercado.

Nuevas oportunidades y amenazas aparecen en ciclos extremadamente cortos.

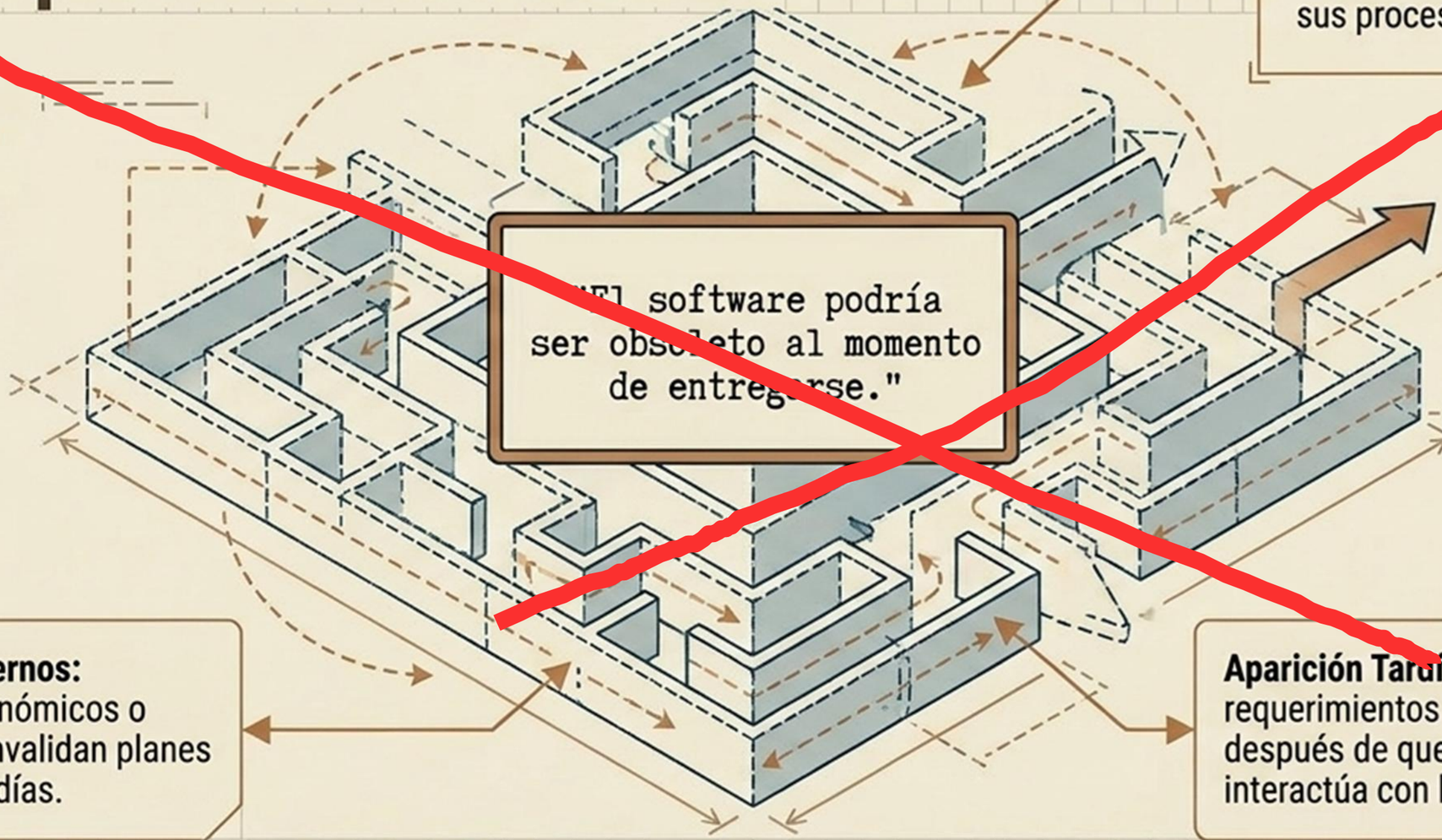


La entrega rápida dejó de ser un plus para convertirse en el requerimiento core de las empresas.

El software es el motor invisible de casi todas las operaciones industriales modernas.

La ilusión de la estabilidad: Los requerimientos mutan

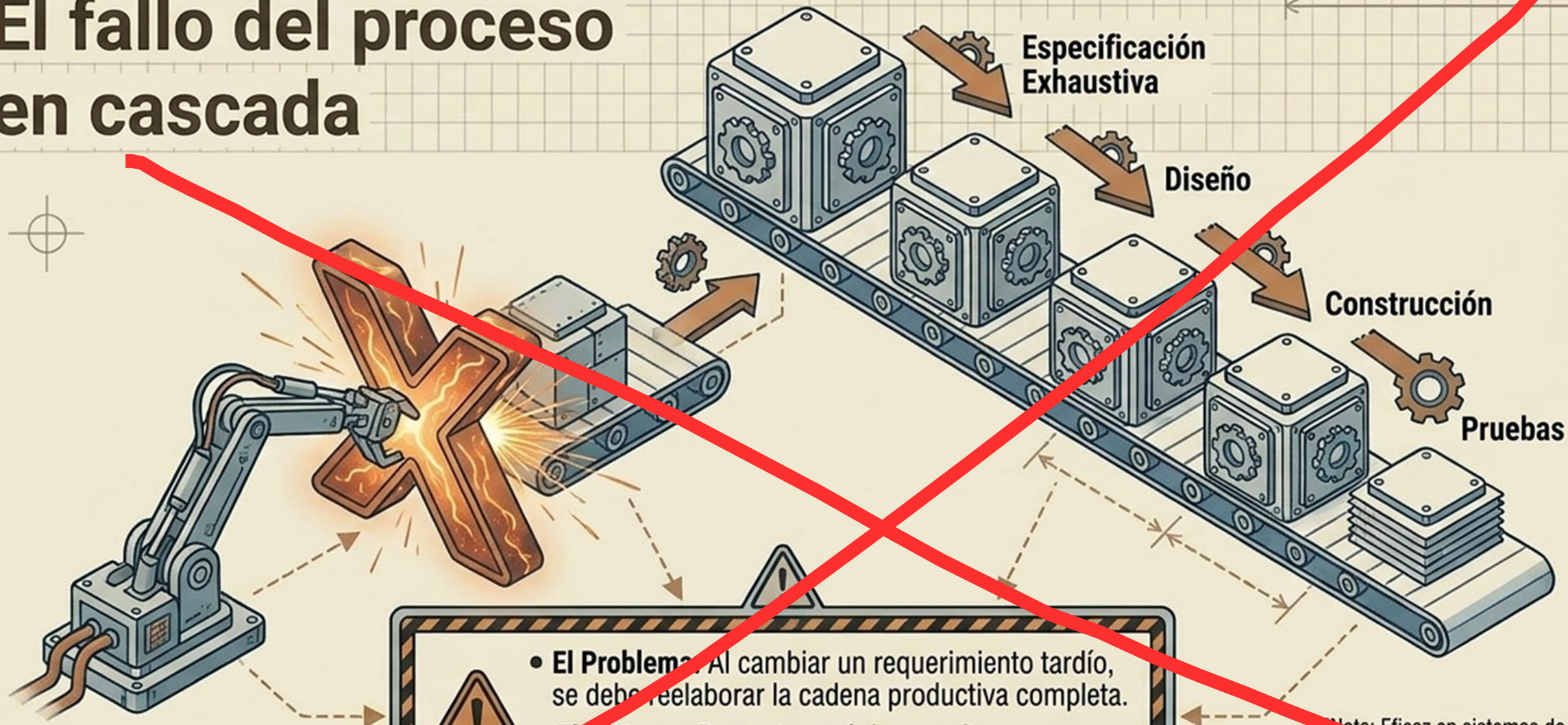
Impredecibilidad: Los clientes no pueden prever el impacto real del sistema en sus procesos diarios.



Factores Externos: Cambios económicos o normativos invalidan planes estáticos en días.

Aparición Tardía: Los verdaderos requerimientos suelen emerger después de que el usuario interactúa con la primera versión.

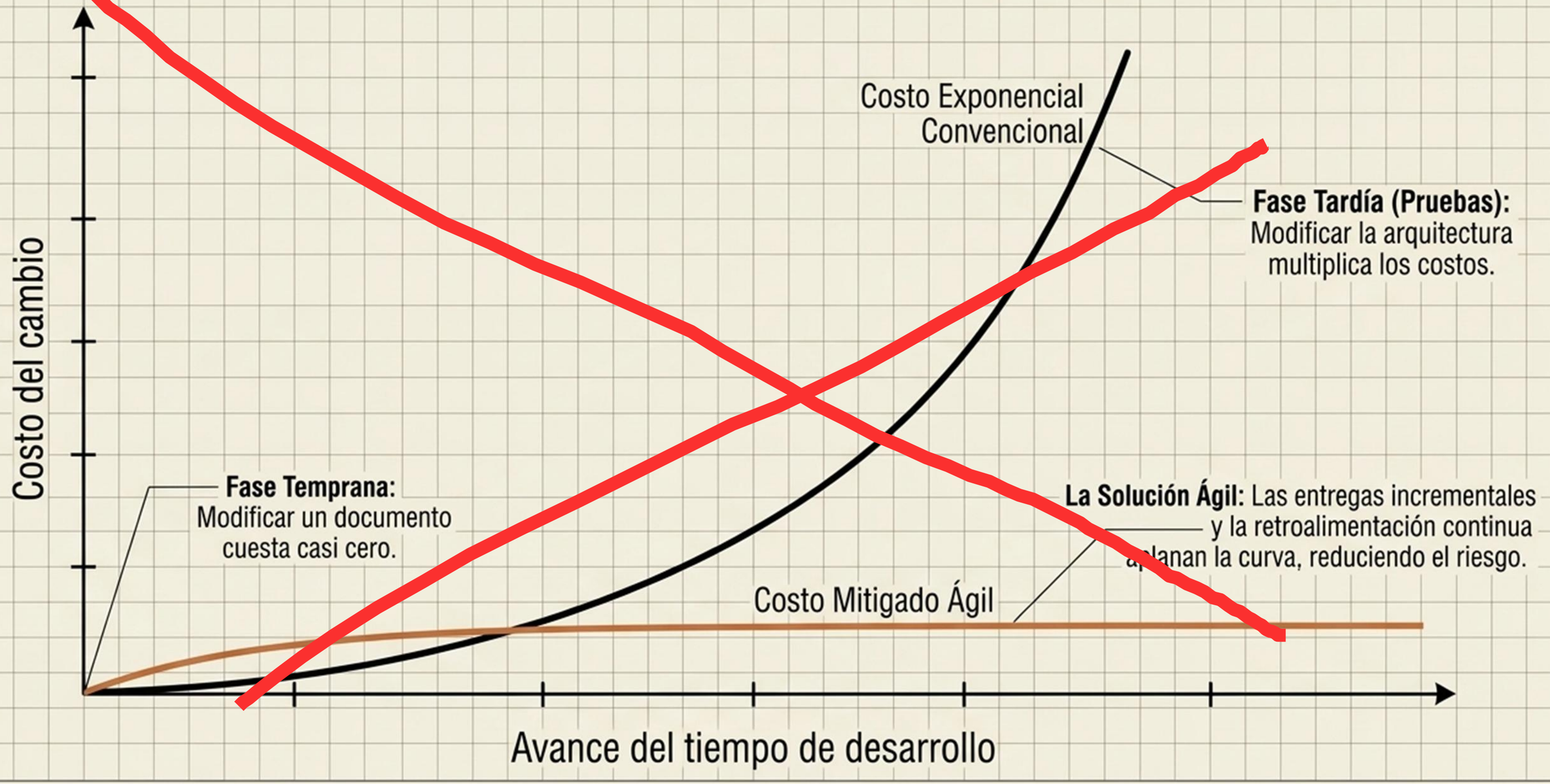
El fallo del proceso en cascada



- **El Problema:** Al cambiar un requerimiento tardío, se debe reelaborar la cadena productiva completa.
- **El Síntoma:** Proyectos crónicos, software que no encaja y fechas de entrega incumplidas.

[Nota: Eficaz en sistemas de análisis crítico previo (ej. aviónica), pero altamente destructivo en software de negocios dinámico.]

La economía del cambio: ¿Por qué cambiar tarde cuesta tanto?

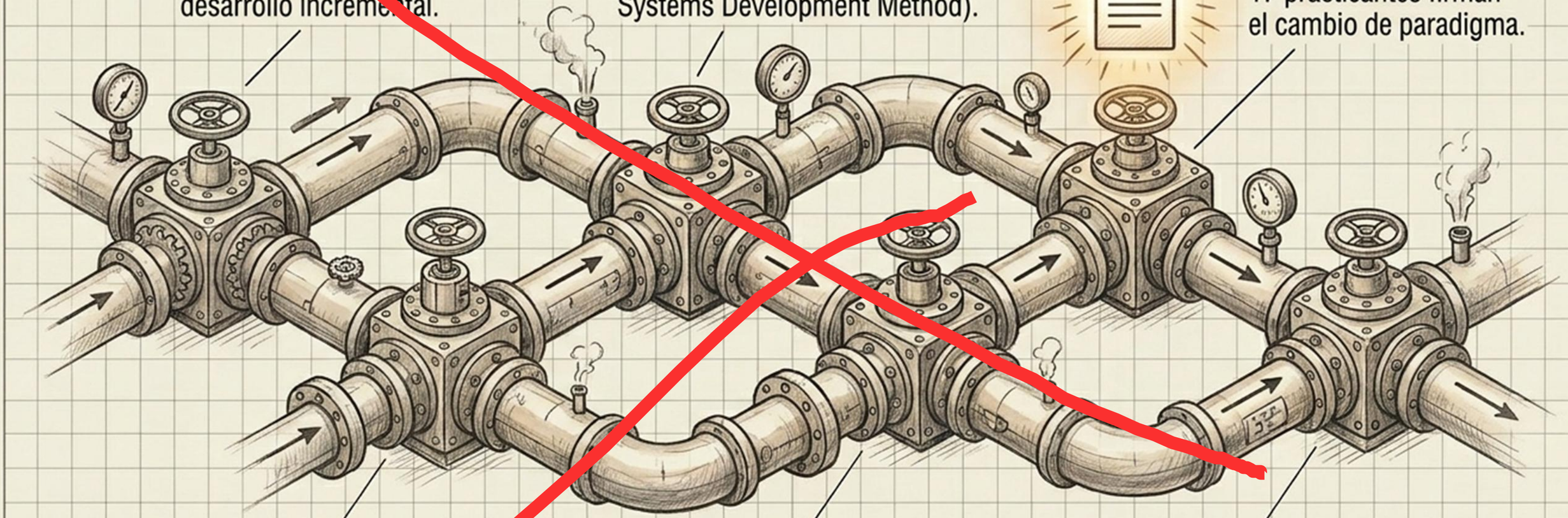


La evolución hacia la adaptabilidad

1980s: IBM introduce el desarrollo incremental.

1997: DSDM (Dynamic Systems Development Method).

2001: El Manifiesto Ágil. 17 practicantes firman el cambio de paradigma.



1990s: Descontento masivo de la industria con metodologías pesadas.

1999-2001: XP (Beck) y Scrum (Schwaber) formalizan la iteración.

Hoy: Un ecosistema consolidado de metodologías.

Manifiesto por el Desarrollo Ágil de Software (2001)

Estamos descubriendo mejores formas de desarrollar software, al hacerlo y al ayudar a otros a hacerlo. Gracias a este trabajo llegamos a valorar:

Individuos e interacciones sobre procesos y herramientas.
Software funcionando sobre documentación exhaustiva.
Colaboración con el cliente sobre negociación de contratos.
Respuesta al cambio sobre seguir un plan.

Aunque hay valor en los elementos de la derecha,
valoramos más los de la izquierda.

Pragmática del Manifiesto: Desglosando los valores (1/2)

Agile (Valoramos más)

Individuos e interacciones

El proceso se adapta a las personas. Un equipo cohesionado supera a un proceso rígido.

Software funcionando

El código funcional es la única medida real de avance.

Traditional (Valoramos, pero...)

Procesos y herramientas

Forzar a personas talentosas en flujos burocráticos mata la eficiencia.

Documentación exhaustiva

La documentación debe ser esencial, evitando crear artefactos que nadie lee.

Pragmática del Manifiesto: Desglosando los valores (2/2)

Agile (Valoramos más)

Colaboración con el cliente

El cliente es un eslabón del equipo diario. Acuerdos flexibles por tiempo/material.

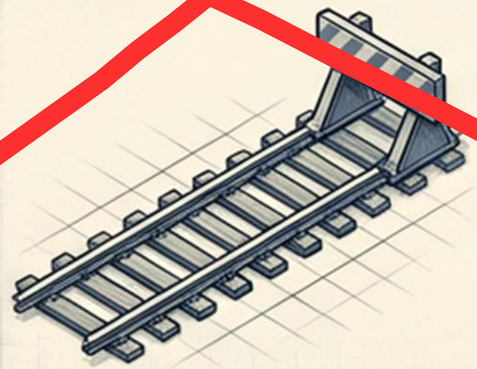
Respuesta al cambio

El plan es una brújula, no un mandato. Iterar permite descubrir la solución correcta.

Traditional (Valoramos, pero…)

Negociación de contratos

El cliente como auditor externo. Los contratos rígidos generan fricción destructiva ante el cambio.



Seguir un plan

Seguir ciegamente un plan inicial en entornos volátiles garantiza entregar exactamente lo incorrecto.

Los 12 principios operativos (La Alianza Ágil)

Orientación al Cliente

1. **Entrega continua** de valor.
2. **2.** Bienvenidos los cambios (ventaja competitiva).
3. **3.** Software funcional en ciclos cortos.
4. **4.** Negocio y desarrolladores unidos a diario.

Dinámica del Equipo

5. **5.** Individuos motivados en entornos de confianza.
6. **6. Comunicación** cara a cara.
7. **7.** Ritmo de trabajo sostenible (cero burnout).
8. **8 . Equipos auto-organizados** (arquitecturas emergentes).

Proceso y Calidad

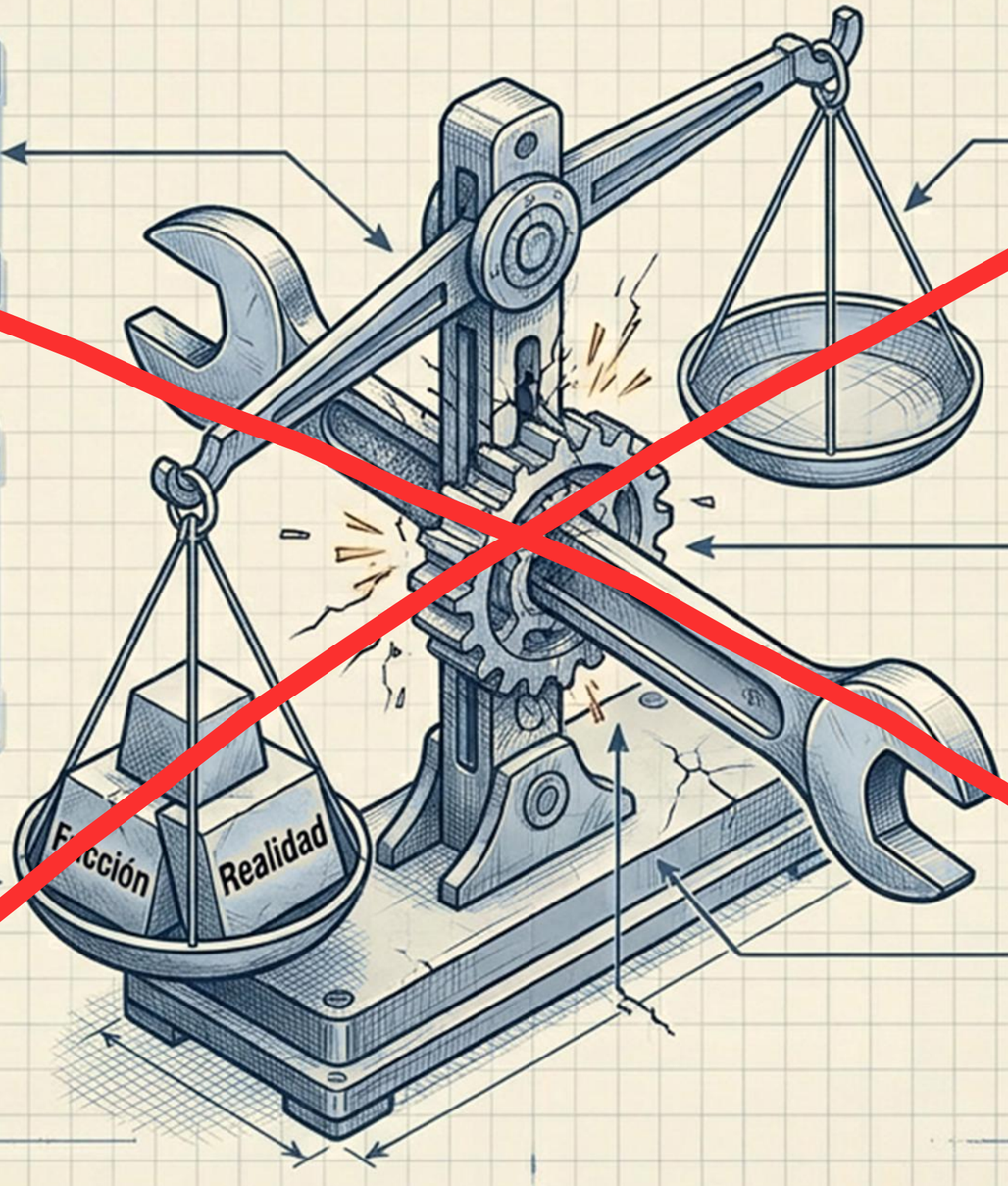
9. **9. Software** operativo como medida base.
10. **10. Excelencia técnica** y diseño continuo.
11. **10. Simplicidad**: Maximizar el trabajo no realizado.
12. **12. Reflexión** y **ajuste** periódico.

UTN FACULTAD REGIONAL RAFAELA

Fricción y realidad: ¿Son siempre aplicables?

Disponibilidad del Cliente:
Frecuentemente no tienen el tiempo para participar a diario.

Perfil del Desarrollador:
La auto-organización requiere una madurez y un talento que no todos poseen.



Caos de Stakeholders:
Priorizar cambios se vuelve político y caótico con múltiples interesados corporativos.

Tiempo vs. Deuda Técnica:
Refactorizar (simplificar) consume tiempo que la gestión rara vez quiere ceder.

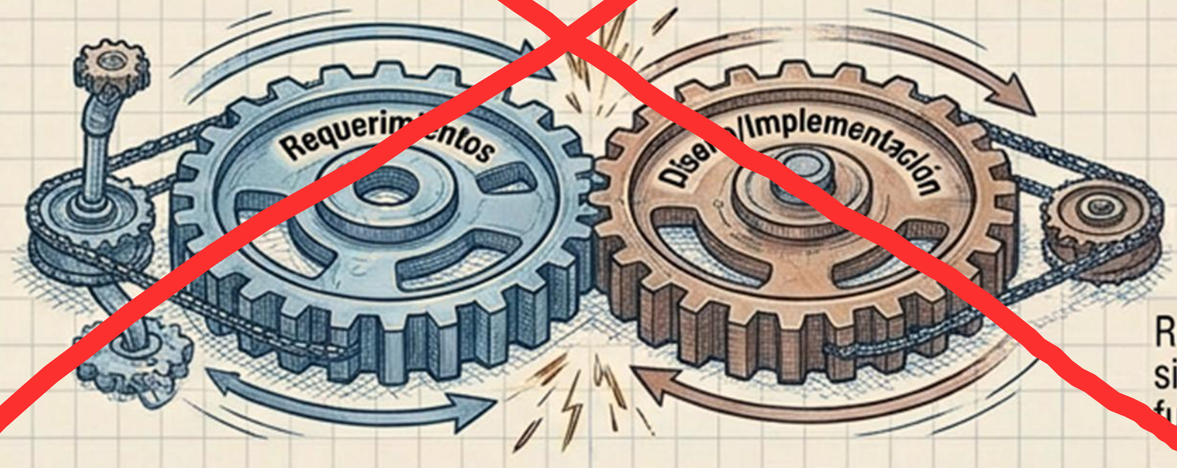
Inercia Corporativa:
Las empresas gigantes resisten abandonar sus procesos de auditoría y validación rígida.

Dos formas de iterar: Ágil vs. Dirigido por un plan

Dirigido por un Plan

El cambio requiere reiniciar bloques enteros desde el inicio.

Ágil

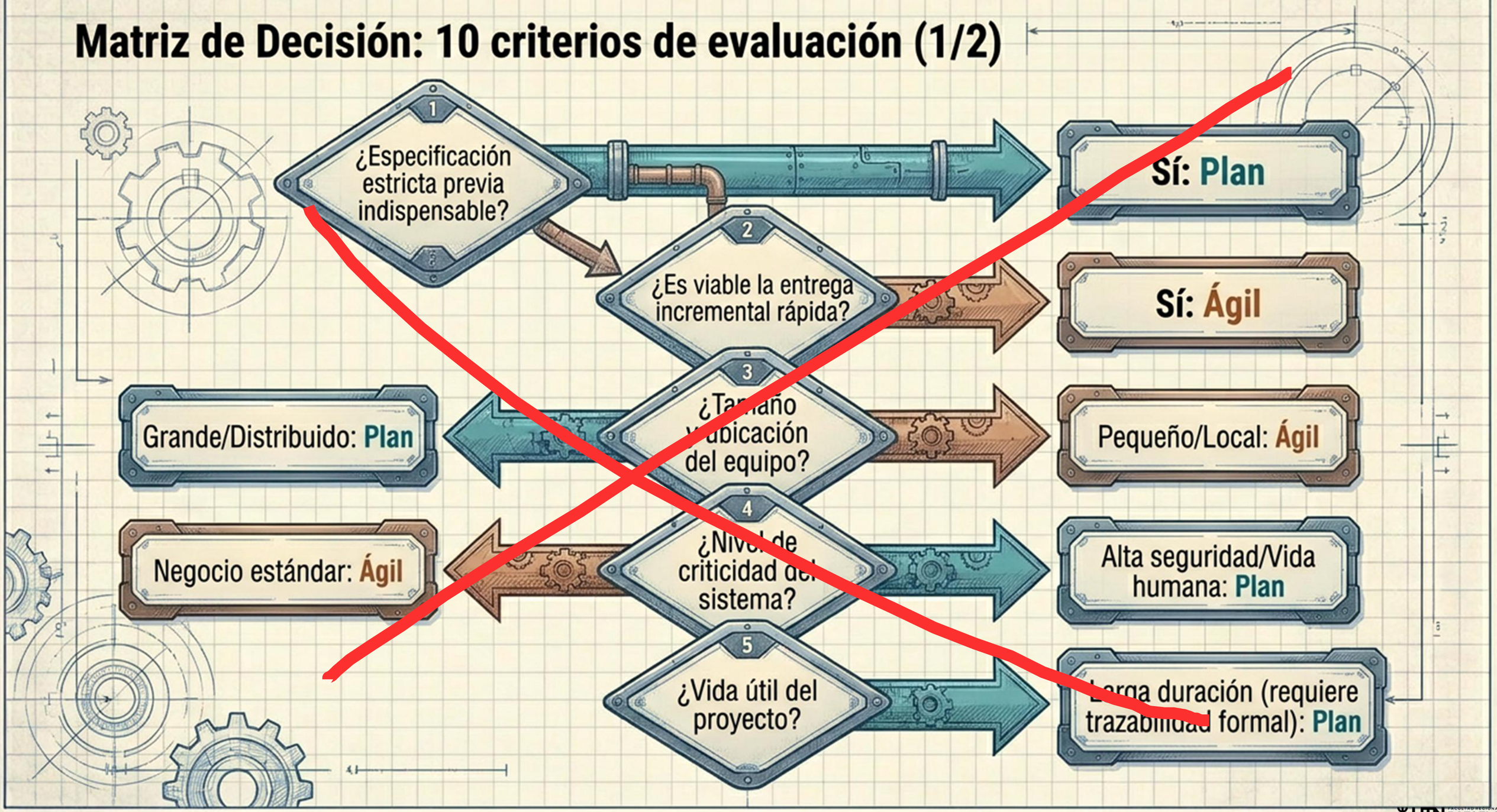


Requerimientos y código evolucionan simultáneamente a través de lotes funcionales rápidos.

Ambos modelos iteran; la diferencia vital radica en cómo y cuándo gestionan la incertidumbre.

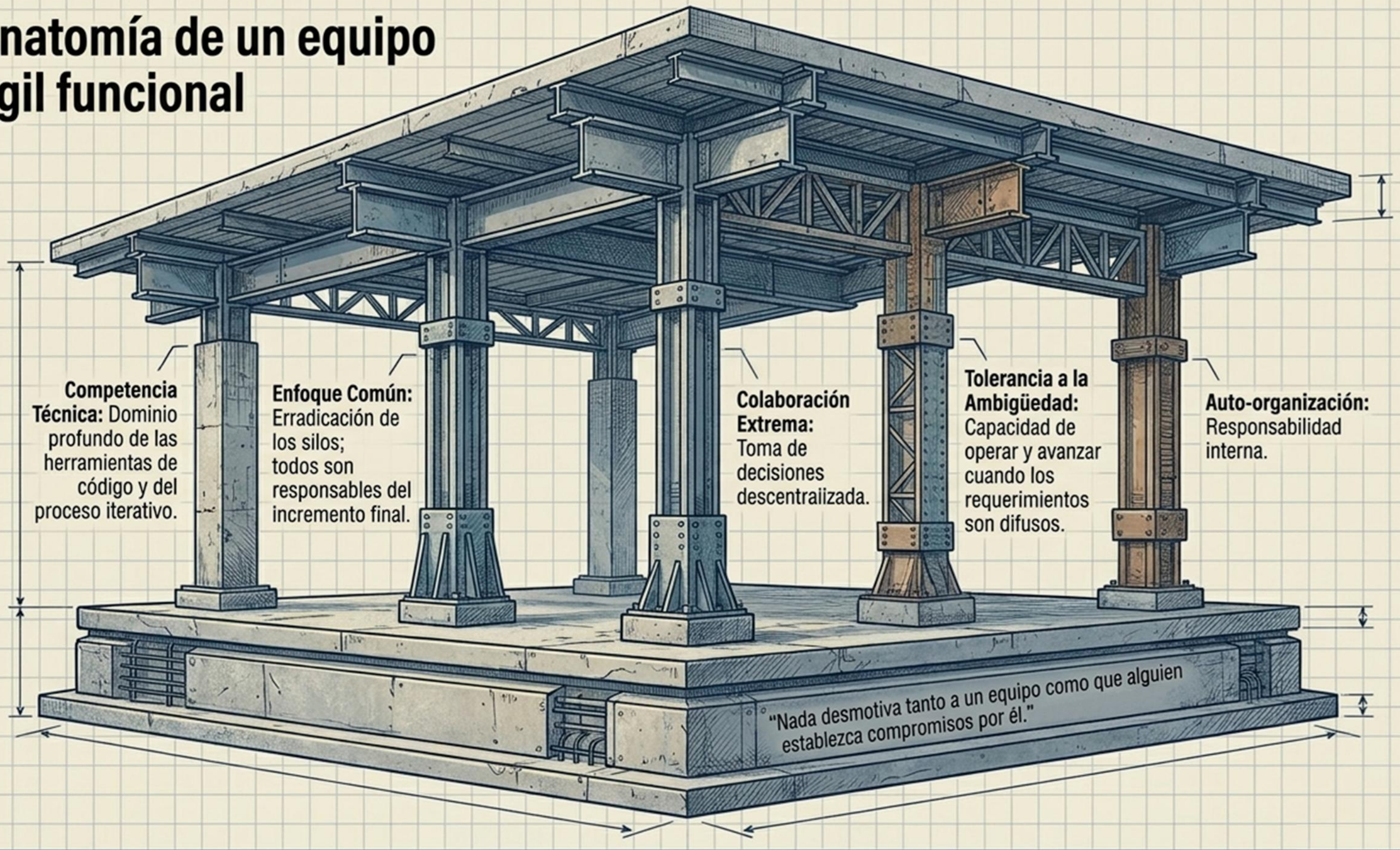
UTN FACULTAD REGIONAL RAFAELA

Matriz de Decisión: 10 criterios de evaluación (1/2)



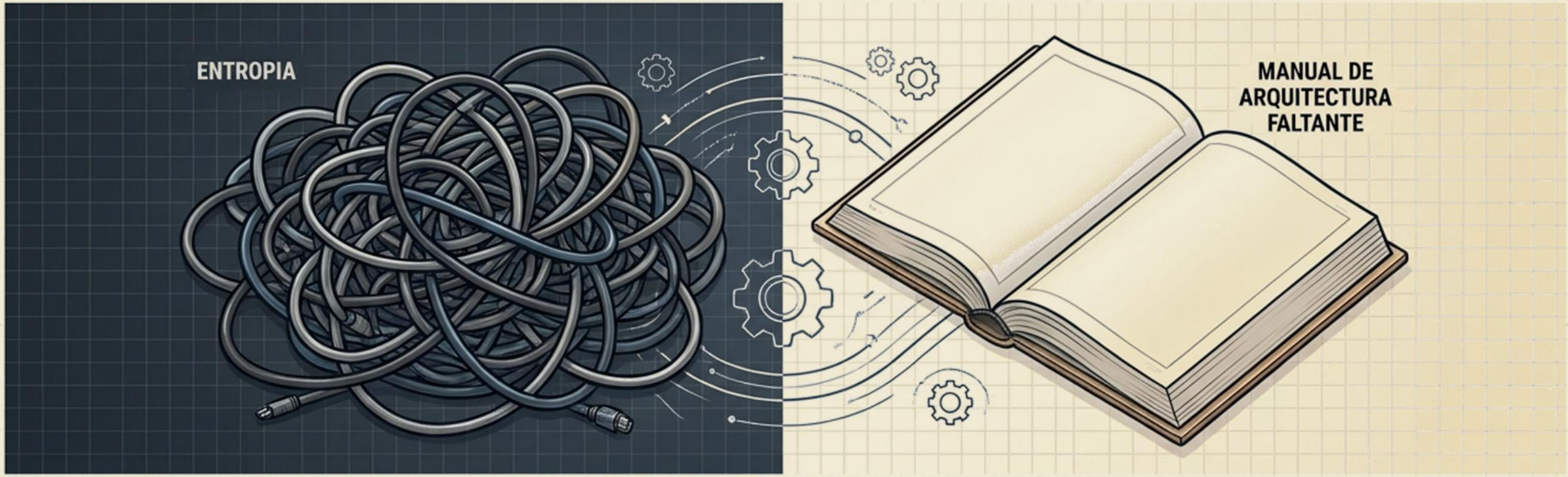
Matriz de Decisión: 10 criterios de evaluación (2/2)

Anatomía de un equipo ágil funcional



UTN RAFAELA

Una pregunta incómoda: ¿Ágil y mantenimiento?



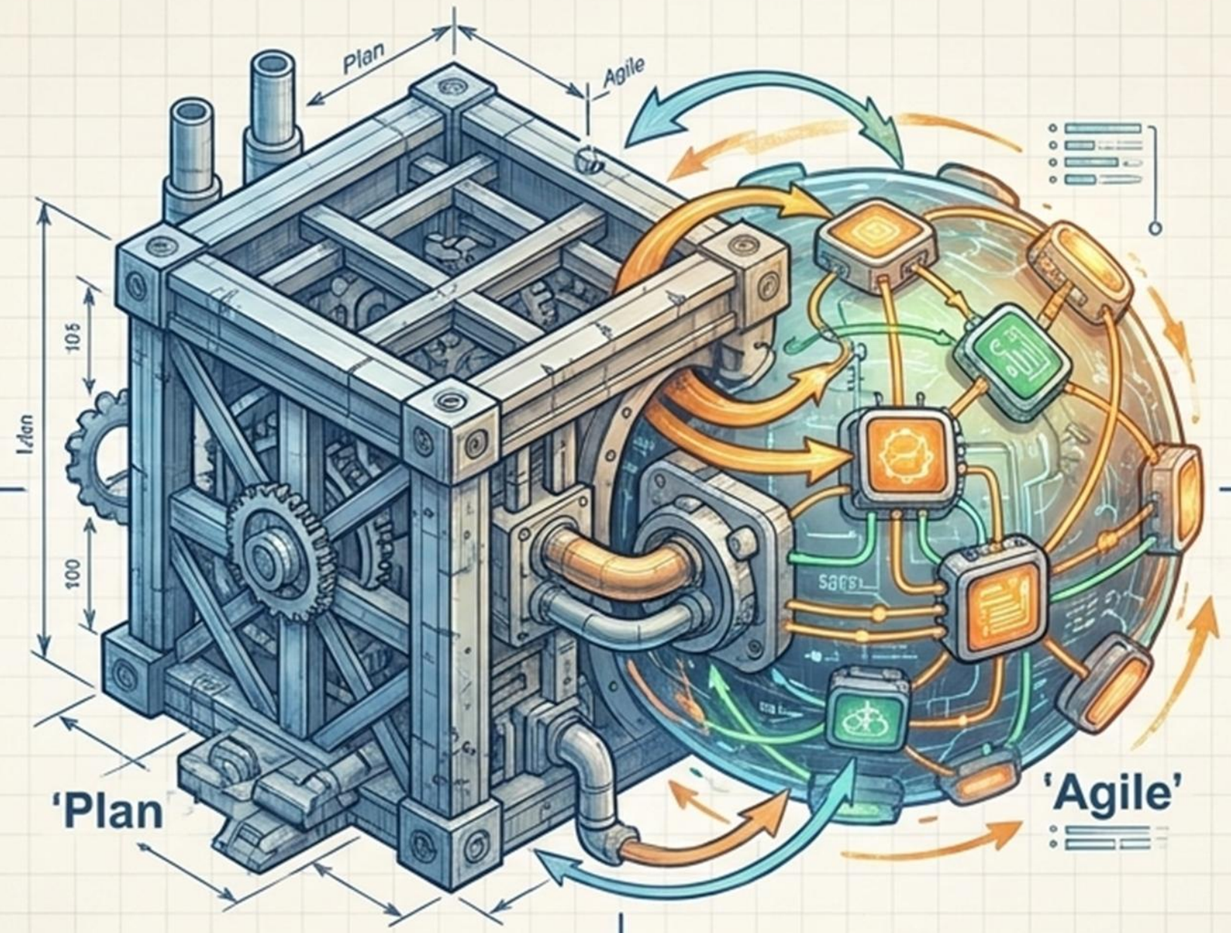
El Mito Ágil: El código limpio y legible reemplaza a la documentación formal.

La Realidad del Software: El 80% del ciclo de vida del software es mantenimiento, no desarrollo de características nuevas.

El Riesgo de Entropía: Si el equipo original (el portador del conocimiento tácito) se disuelve o rota, mantener un sistema sin documentación arquitectónica se vuelve propenso a errores y extremadamente costoso.

El enfoque híbrido: Evitando los dogmas religiosos

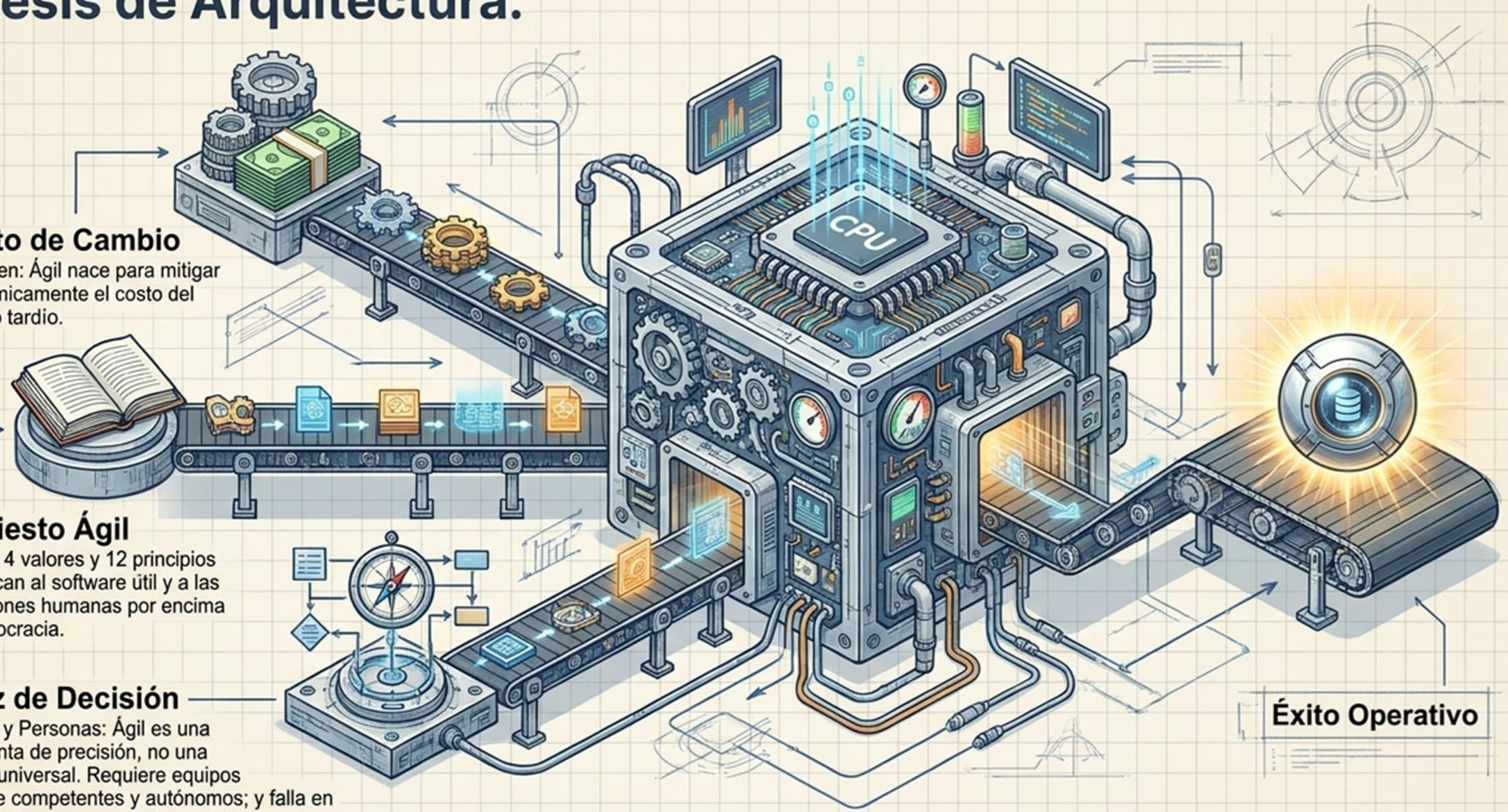
Fin del debate binario: La ingeniería de software de alto nivel no elige entre Ágil o Cascada.



Fusión Operativa: Los proyectos de mayor éxito integran el rigor arquitectónico preventivo de los métodos dirigidos por plan con la ejecución iterativa y adaptativa de los métodos ágiles.

La Pregunta Correcta: Ya no es '¿Debemos usar Ágil?', sino '¿Qué prácticas ágiles específicas maximizan el valor dadas las restricciones de este proyecto?'.

Síntesis de Arquitectura:



Costo de Cambio

El Origen: Ágil nace para mitigar económicamente el costo del cambio tardio.

Manifiesto Ágil

Filosofía: 4 valores y 12 principios que colocan al software útil y a las interacciones humanas por encima de la burocracia.

Matriz de Decisión

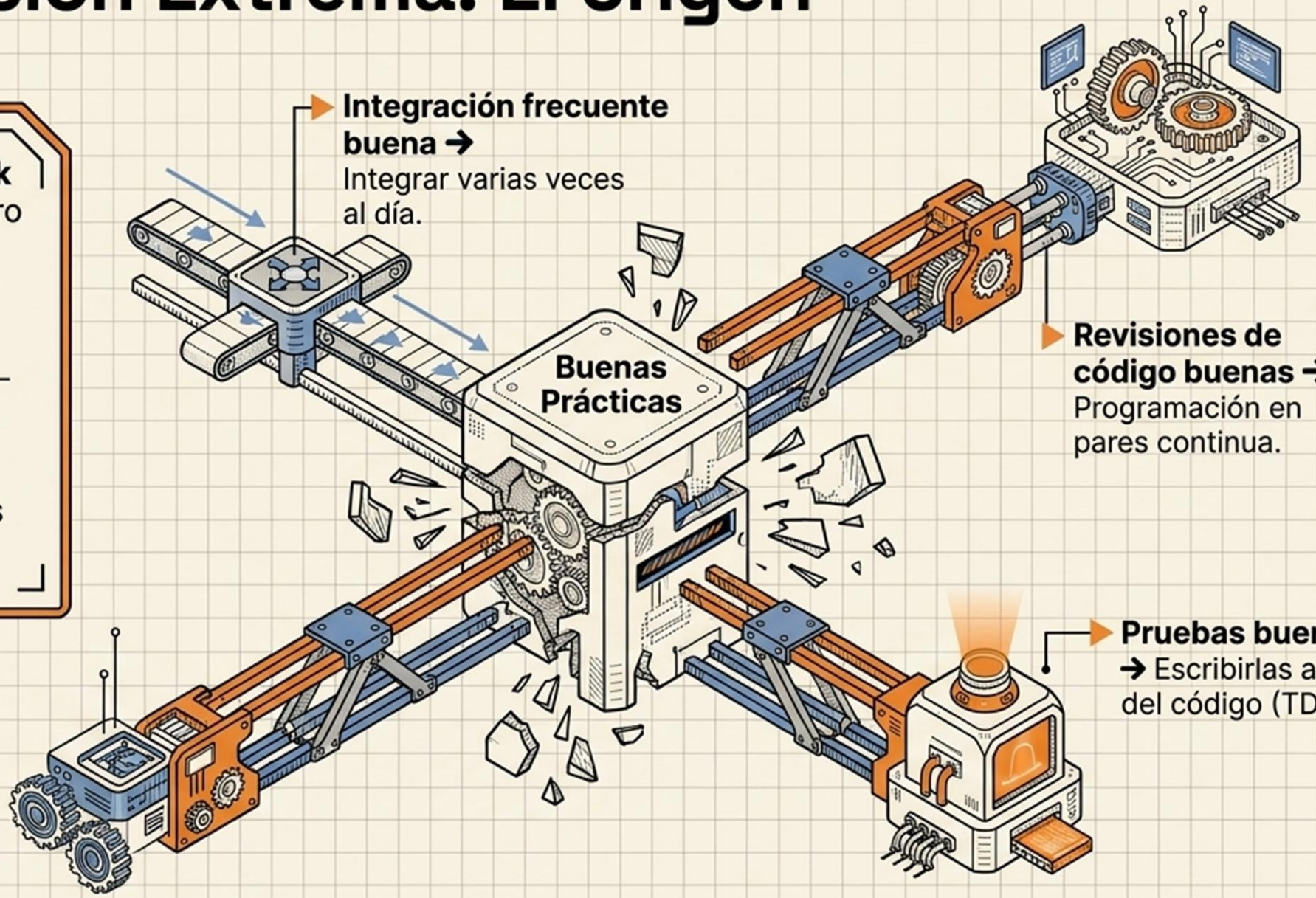
Contexto y Personas: Ágil es una herramienta de precisión, no una solución universal. Requiere equipos altamente competentes y autónomos; y falla en entornos altamente regulados o rigidos.

Programación Extrema (XP)

Programación Extrema: El origen

Creada por **Kent Beck** a fines de los 90 (Libro fundacional: 2000). Enfoque orientado a objetos.

El concepto central: Llevar prácticas reconocidas a niveles “extremos”.

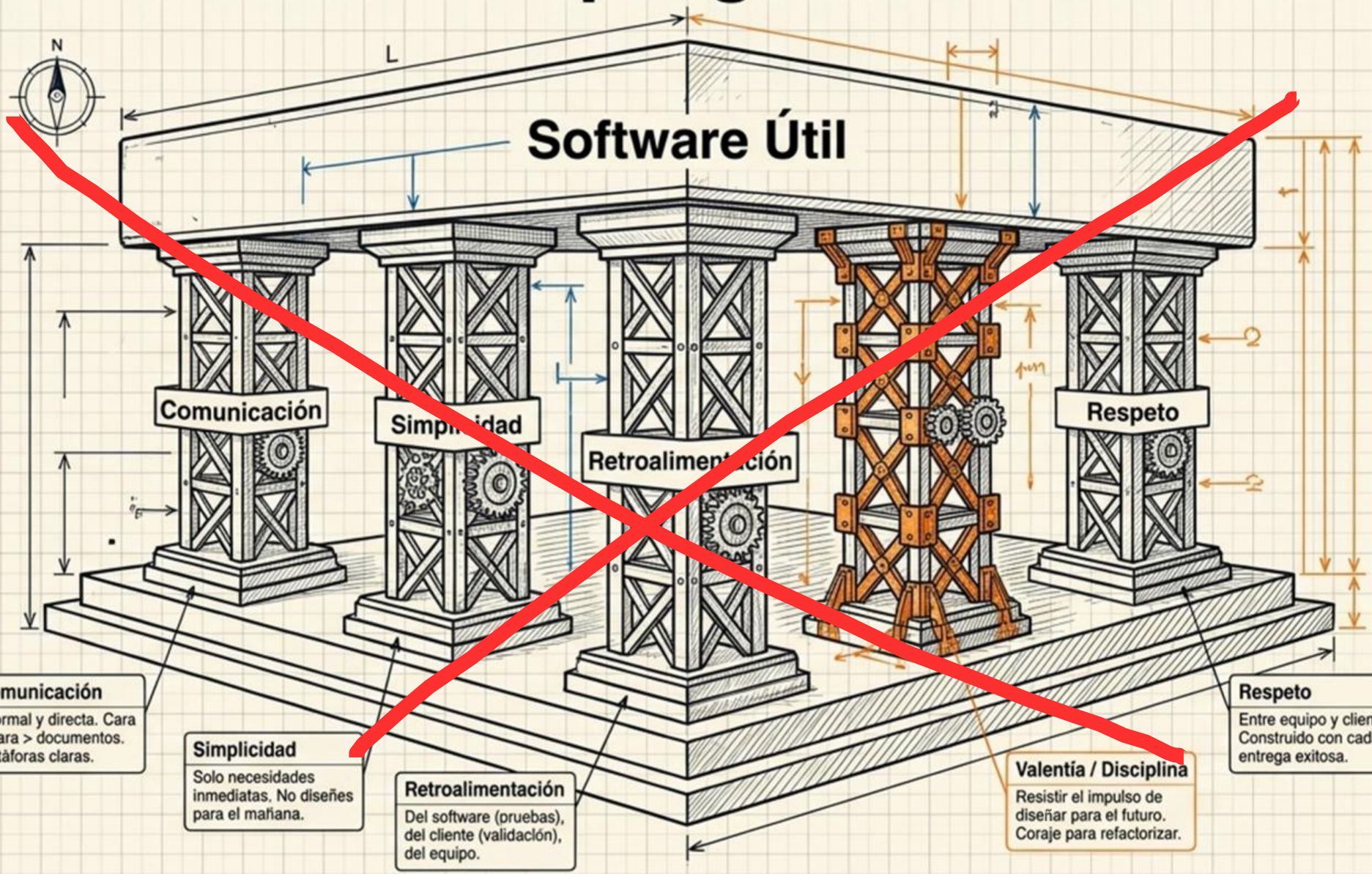


Integración frecuente buena → Integrar varias veces al día.

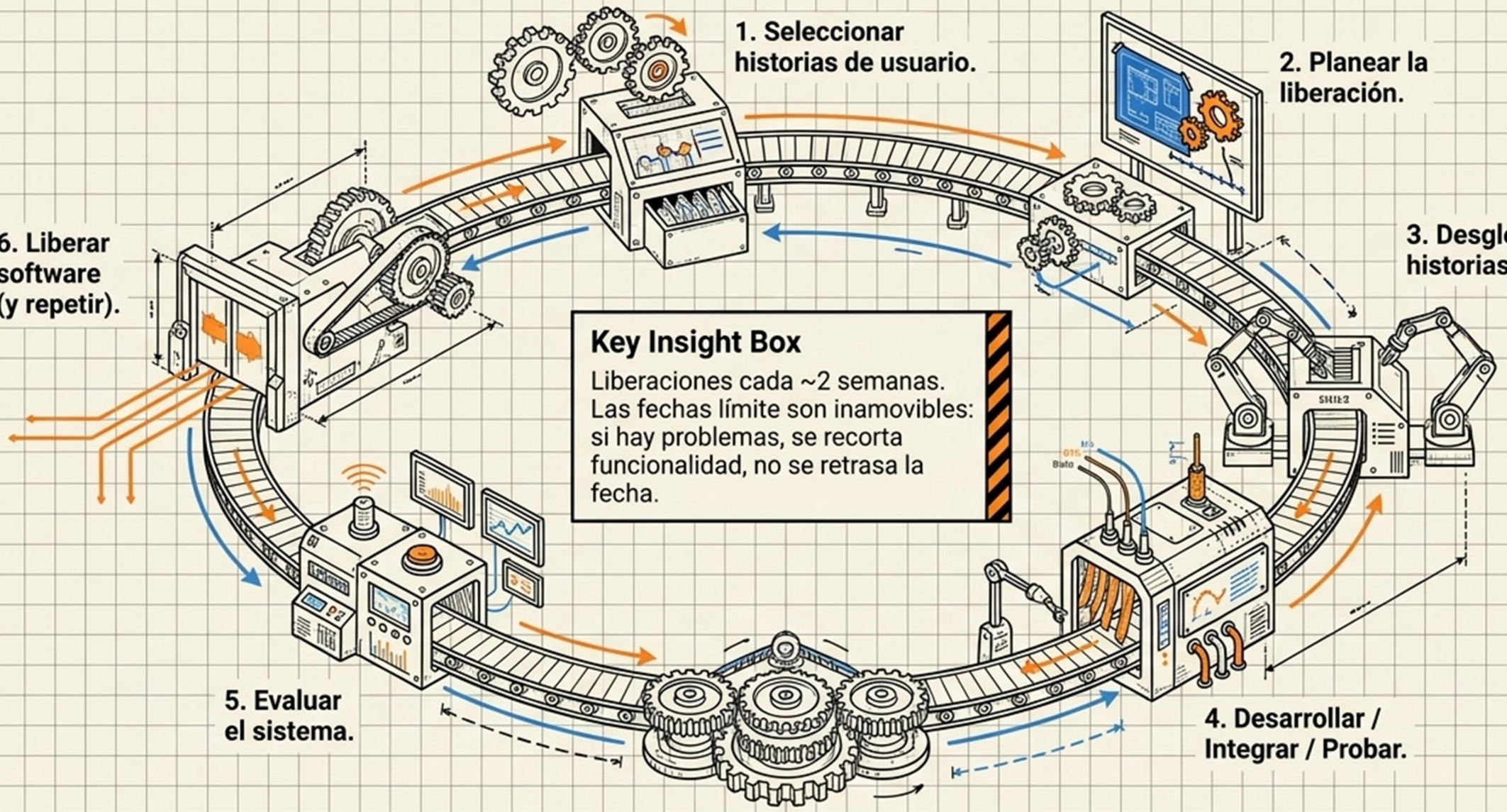
Revisiones de código buenas → Programación en pares continua.

Pruebas buenas → Escribirlas antes del código (TDD).

Los 5 valores que guían todo en XP



¿Cómo funciona el proceso XP?



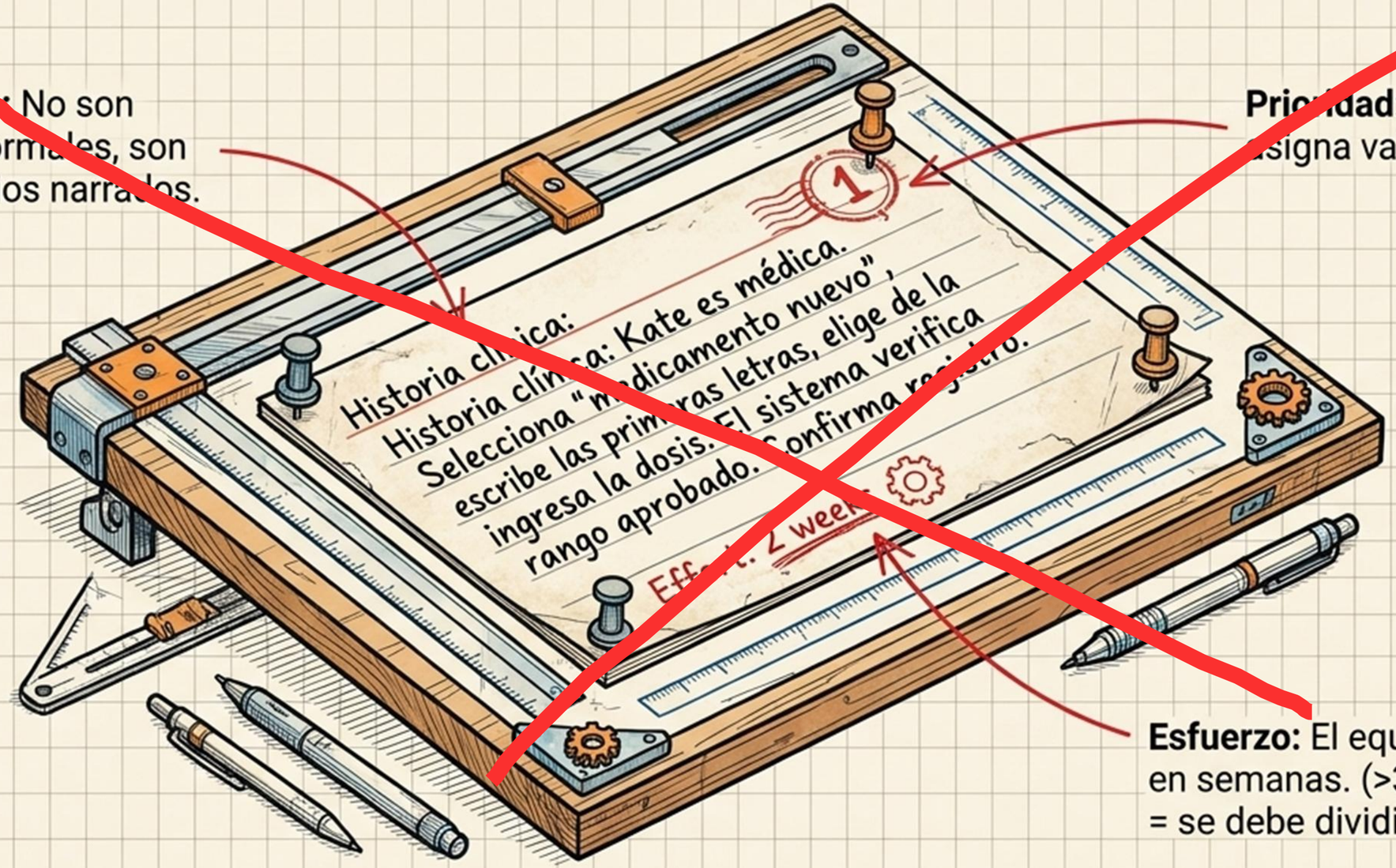
Key Insight Box

Liberaciones cada ~2 semanas. Las fechas límite son inamovibles: si hay problemas, se recorta funcionalidad, no se retrasa la fecha.

Las historias de usuario: el corazón de XP

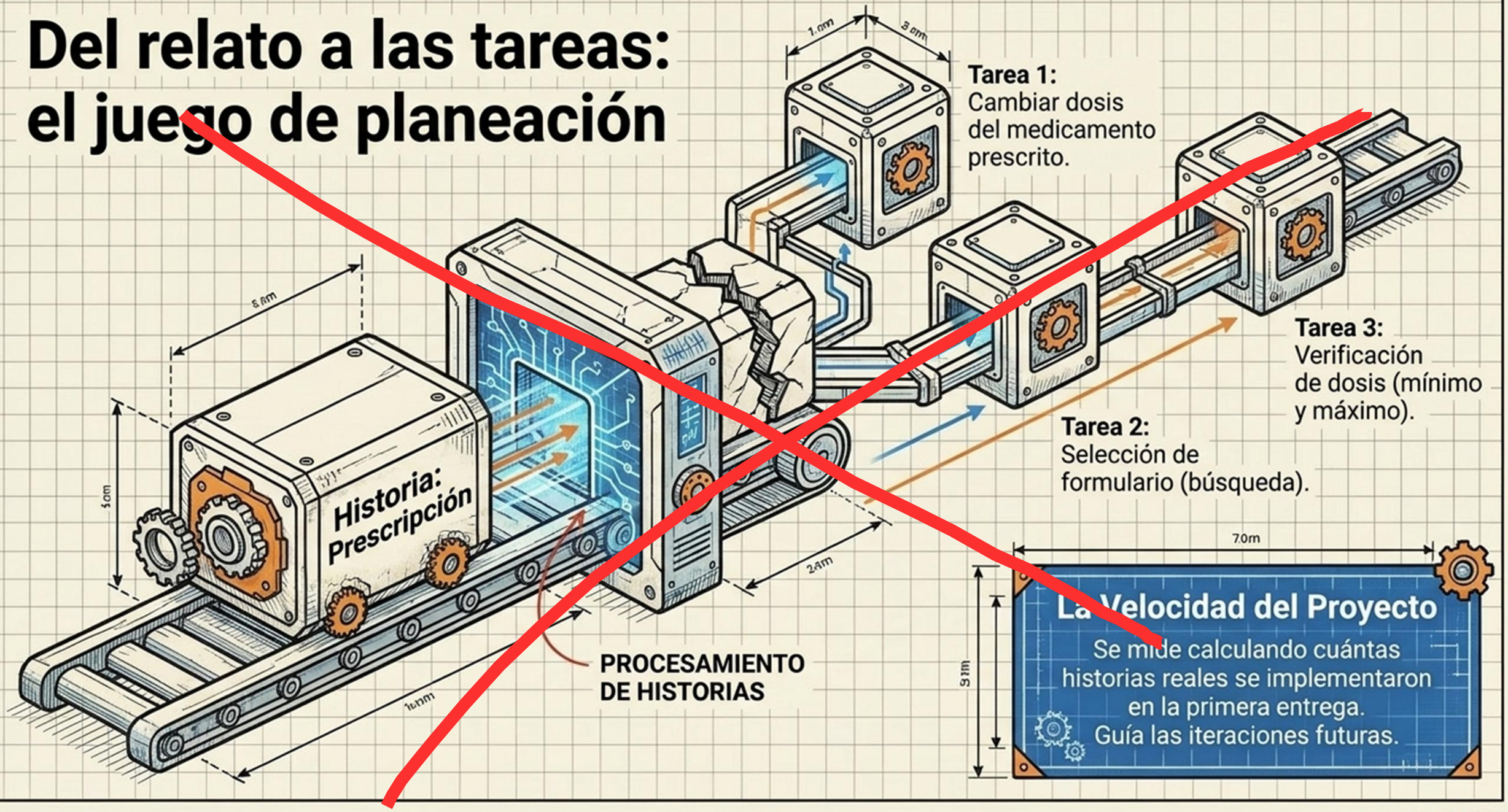
Formato: No son specs formales, son escenarios narrados.

Prioridad: El cliente asigna valor de negocio.



Esfuerzo: El equipo estima en semanas. (>3 semanas = se debe dividir).

Del relato a las tareas: el juego de planeación



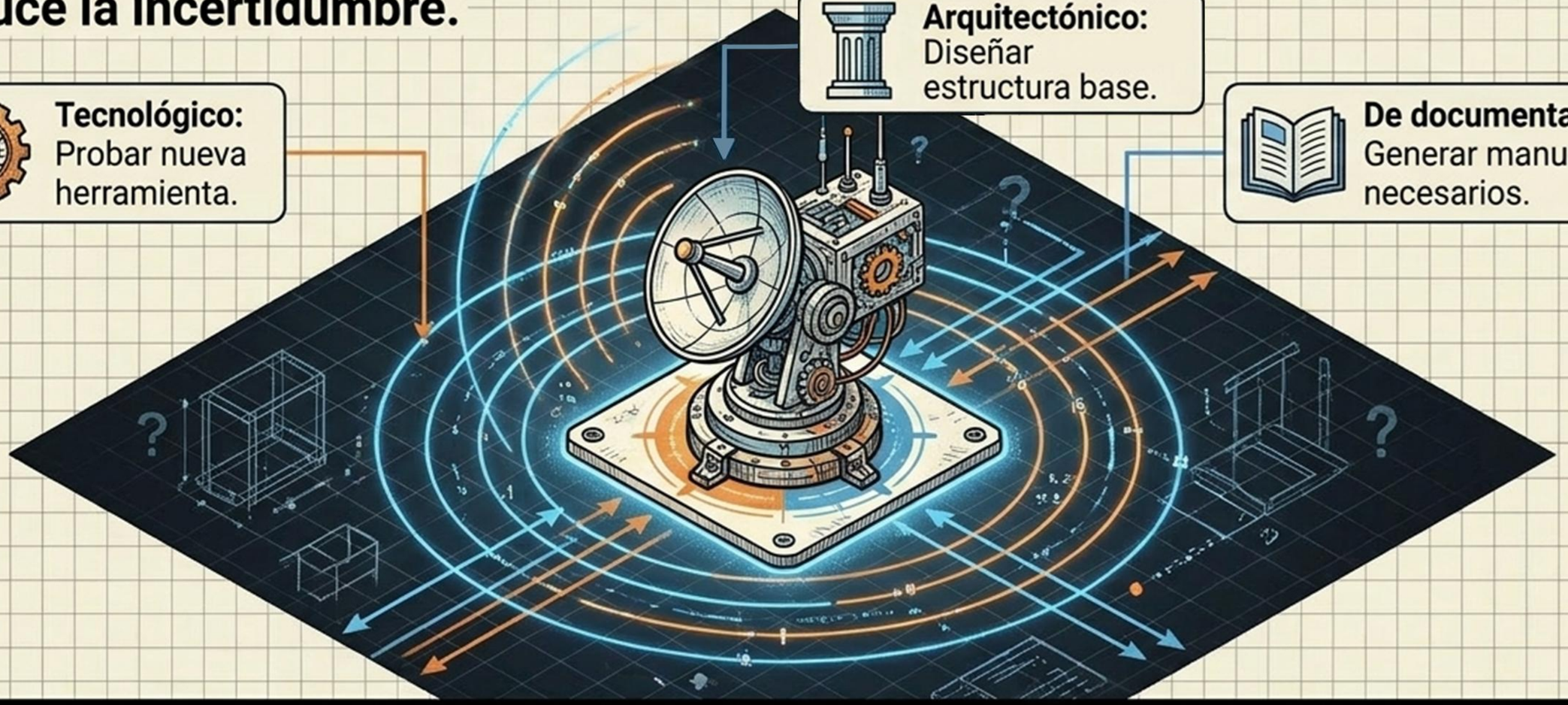
El pico (Spike): explorar antes de comprometerse

Un incremento de exploración. No produce código productivo, reduce la incertidumbre.

Arquitectónico: Diseñar estructura base.

Tecnológico: Probar nueva herramienta.

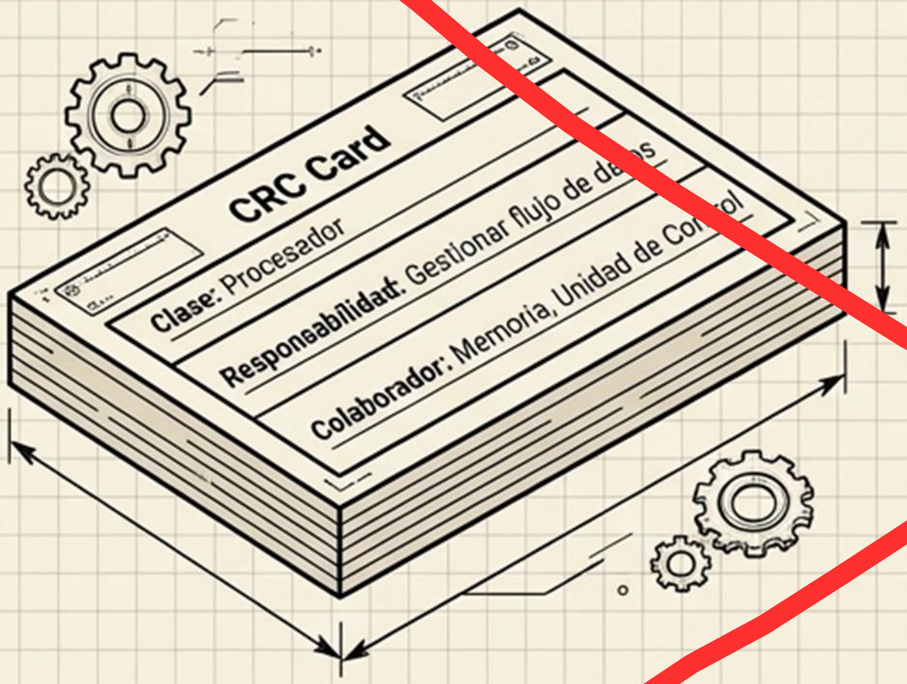
De documentación: Generar manuales necesarios.



Es el reconocimiento de que XP requiere planificación adaptativa. Cuando no sabes, exploras.

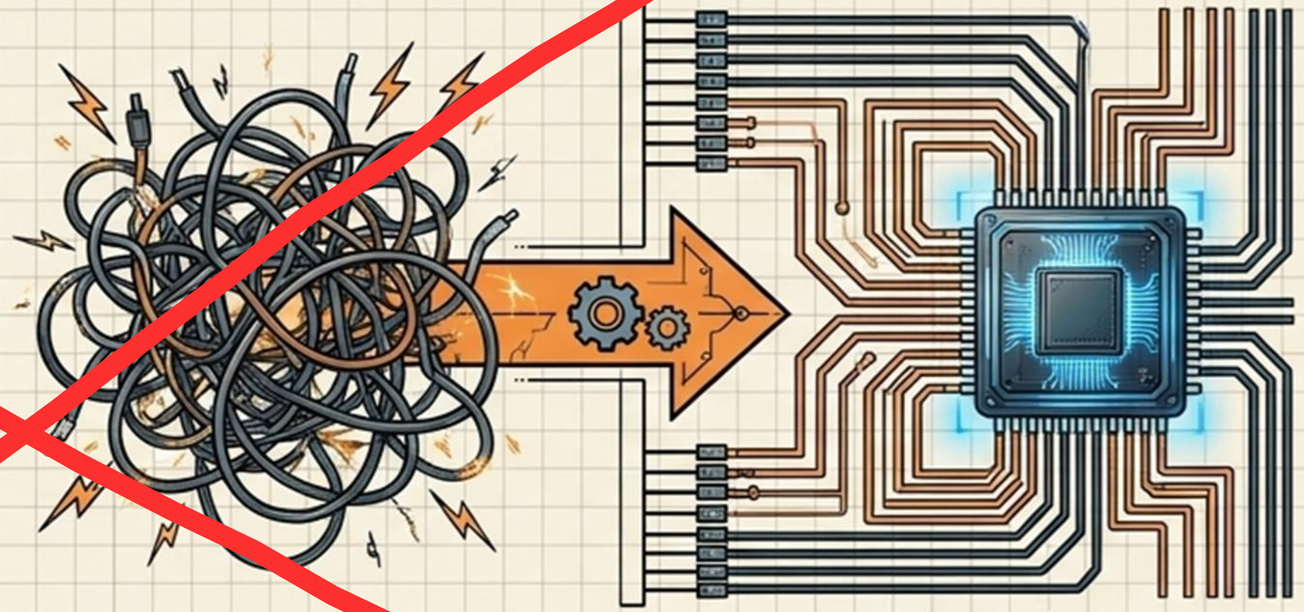
Diseño simple + Refactorización continua

Diseño Simple



Diseñar solo lo necesario hoy.
Herramienta clave: Tarjetas CRC.
No diseñes para el mañana.

Refactorización



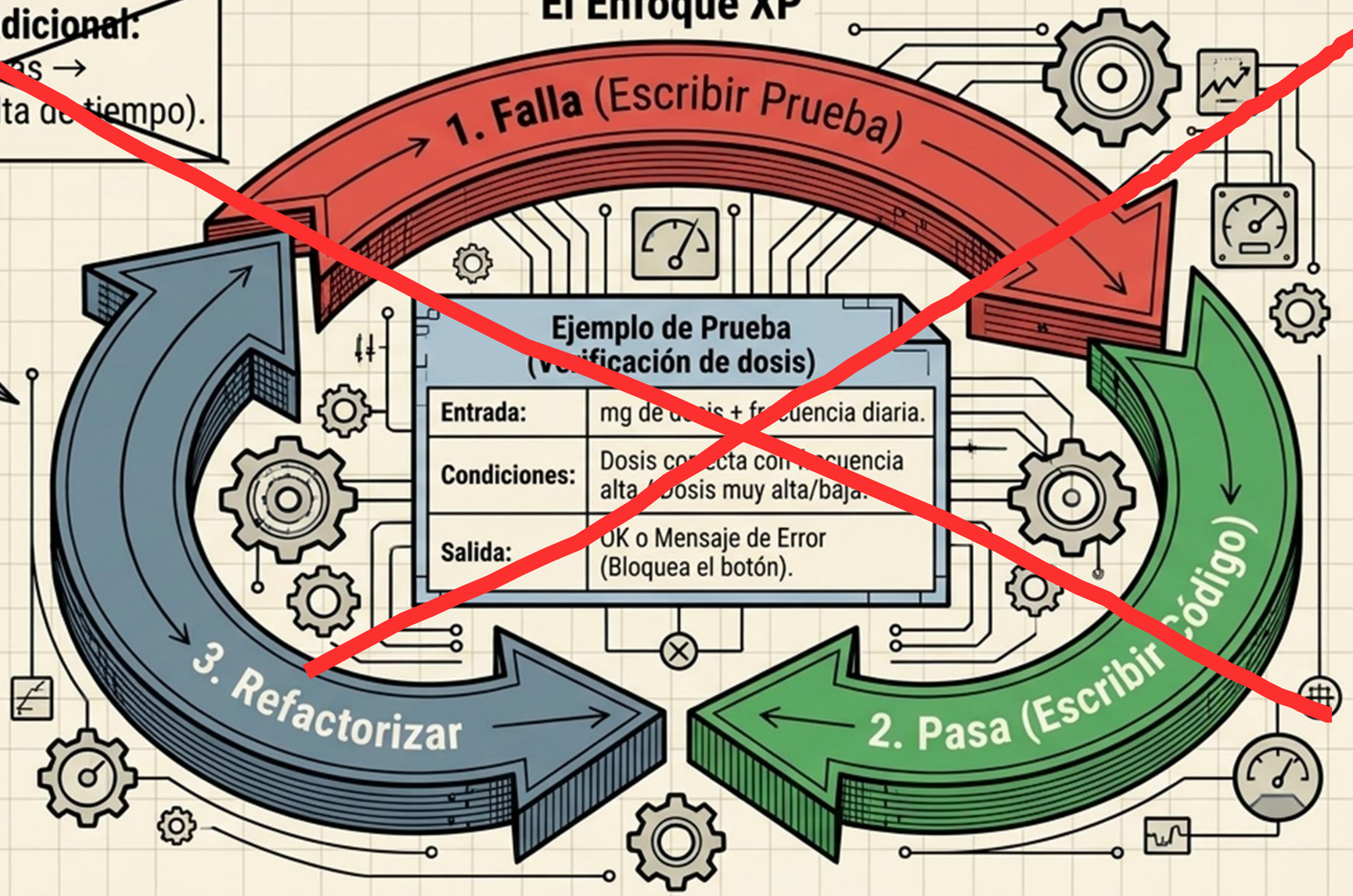
Antes:
Caos (Entropía)

Después:
Orden (Solución Robusta)

Mejorar estructura interna sin cambiar comportamiento externo.
Se hace de forma continua.
Previene la degradación del código.

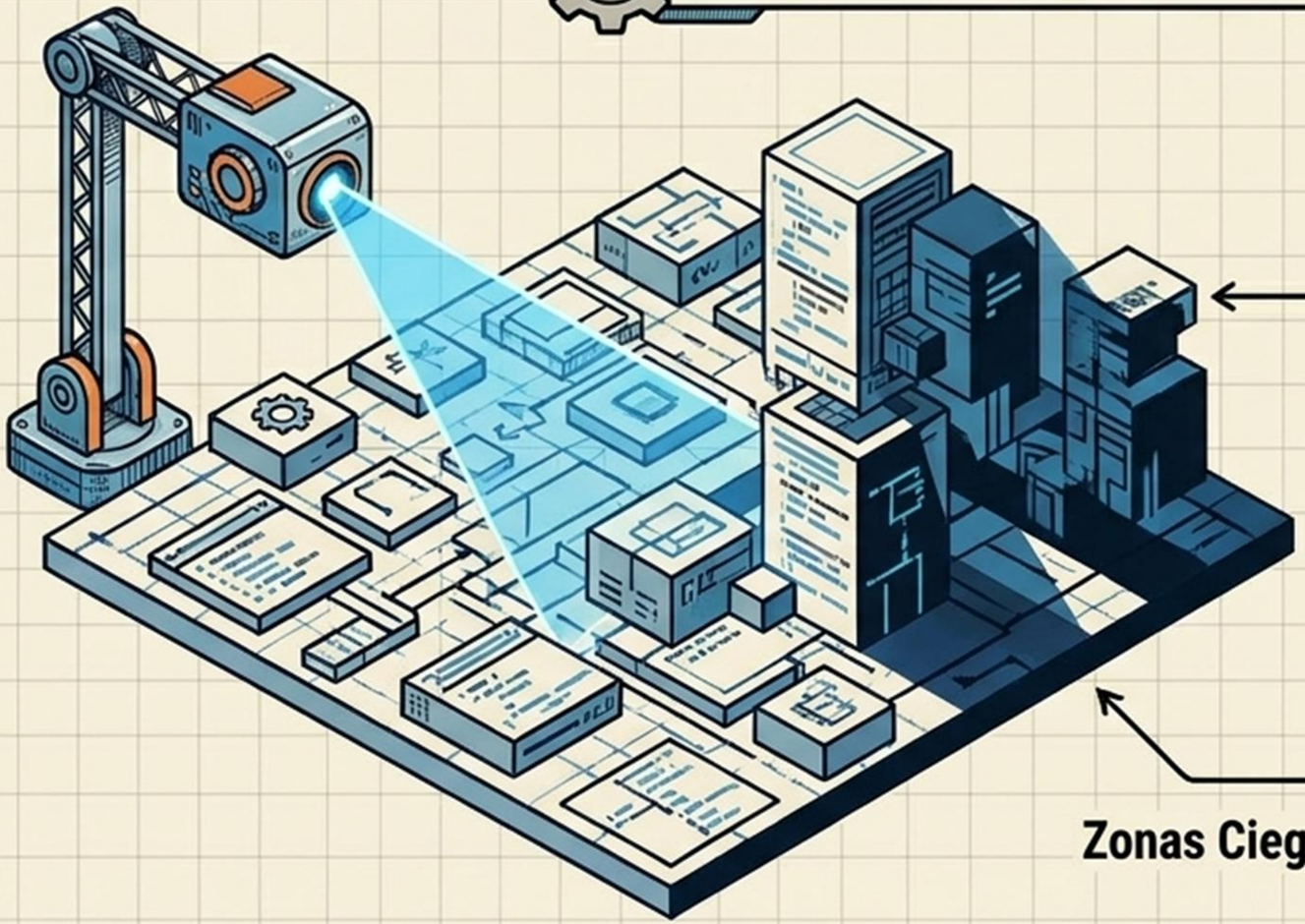
TDD: primero las pruebas, después el código

El Enfoque Tradicional:
Código → Pruebas →
(Omisión por falta de tiempo).



¿Las pruebas automatizadas garantizan calidad?

La Promesa: Ejecutar pruebas da confianza total.



Zonas Ciegas

1. Atajos de programadores: Pruebas incompletas para ahorrar tiempo.

2. Dificultad técnica: Interfaces de usuario (UI) complejas y flujos visuales son difíciles de probar incrementalmente.

3. Ilusión de completitud: Tener muchas pruebas no garantiza que las partes críticas estén cubiertas.

Conclusión: Las pruebas son necesarias pero no suficientes; requieren revisión humana y criterio.

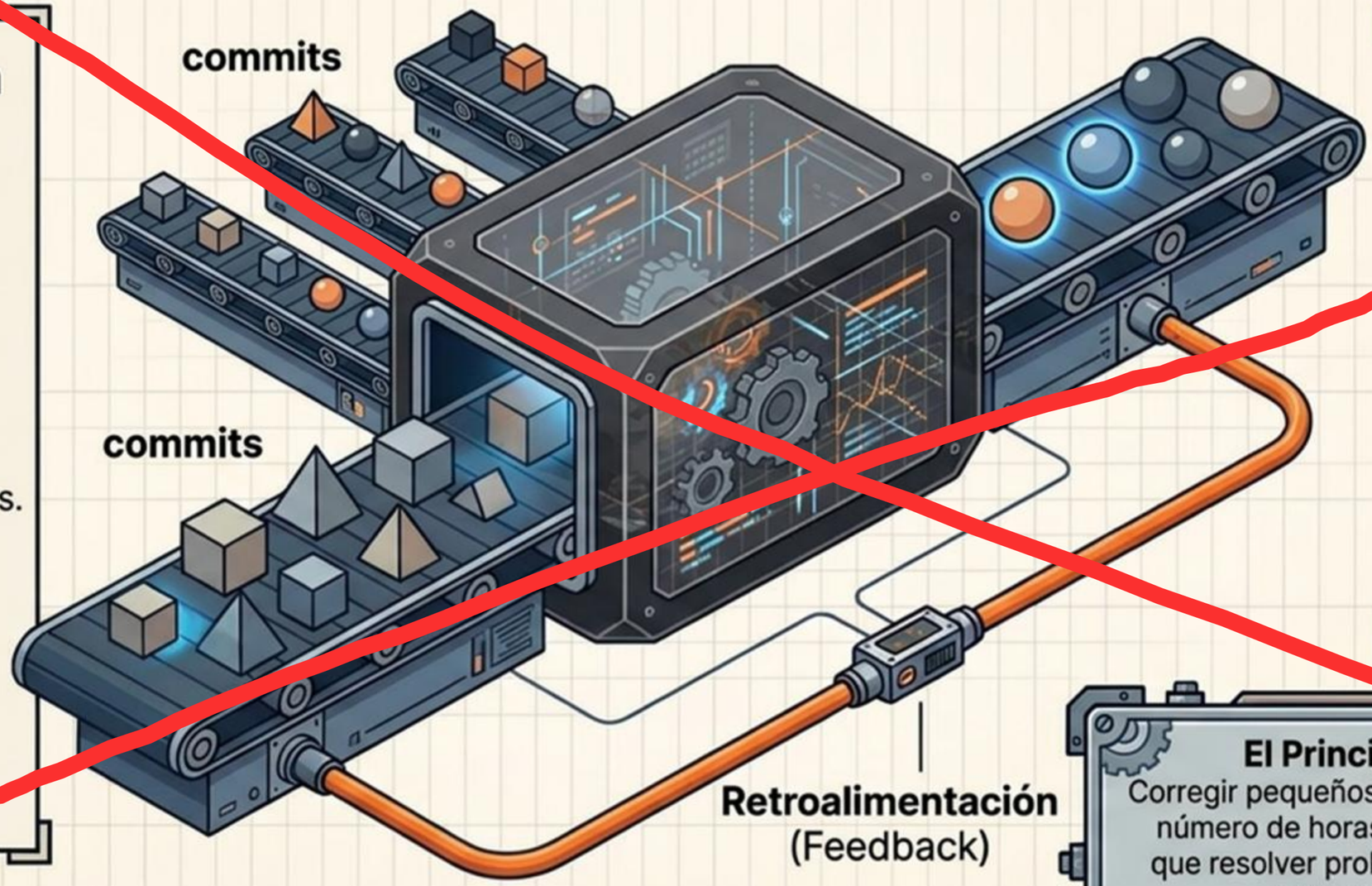
Integración continua + liberaciones pequeñas

Integración Continua

Cada tarea completada se integra de inmediato.

Deben pasar TODAS las pruebas unitarias.

Previene la deuda técnica y el infierno de integración final.



Liberaciones Pequeñas

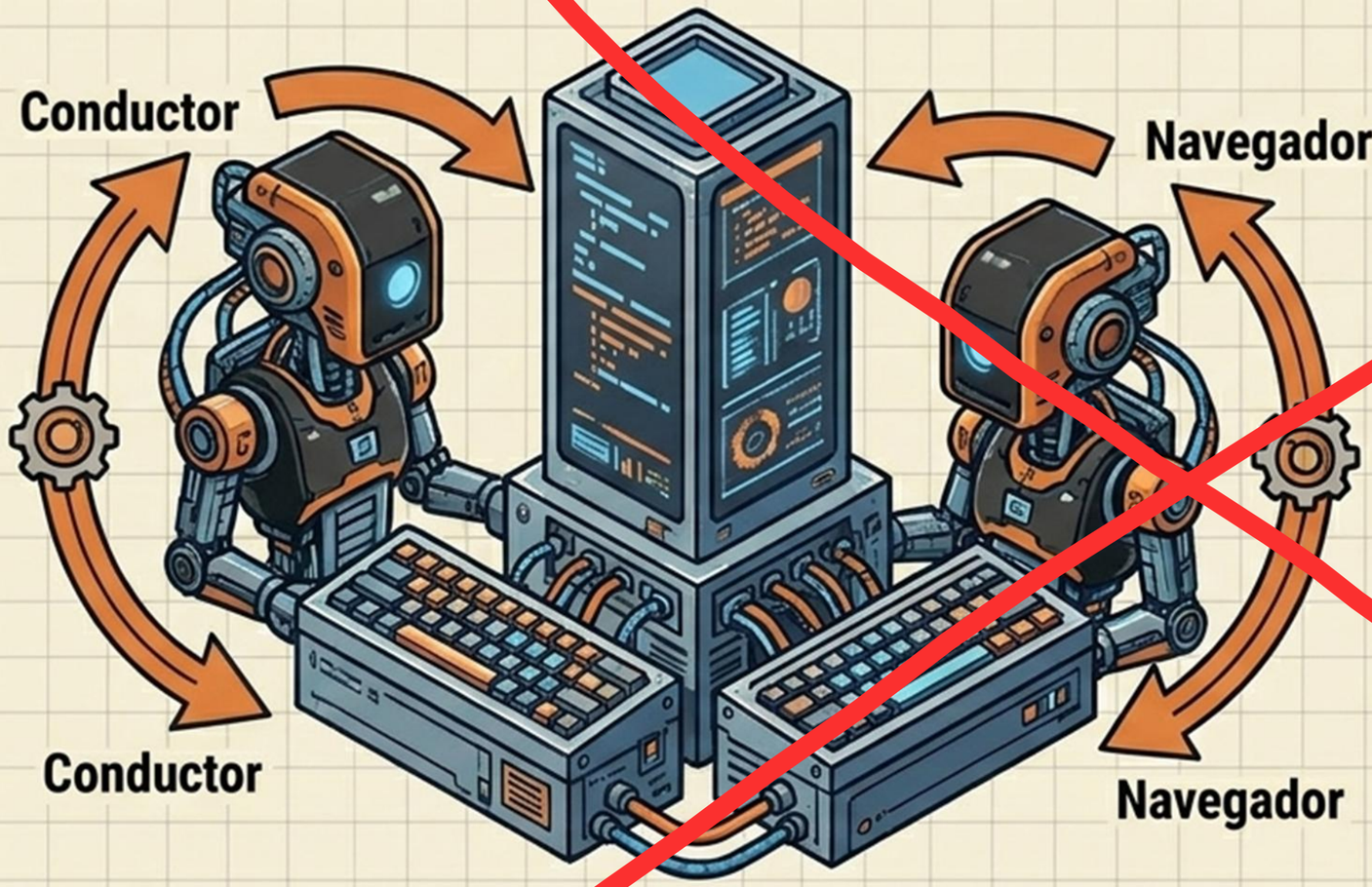
Entregas frecuentes (cada ~2 semanas).

El cliente evalúa y corrige el rumbo rápidamente.

Retroalimentación
(Feedback)

El Principio (Wells) :
Corregir pequeños problemas cada cierto número de horas toma menos tiempo que resolver problemas enormes justo antes del plazo final.

Programación en pares: dos personas, una estación



Mecánica. Roles rotativos. Uno escribe el detalle, el otro verifica estándares y visión global.

3 Ventajas Clave

1. Propiedad colectiva del código.
2. Revisión informal continua.
3. Facilita la refactorización sin fricciones.

El Debate de Productividad

¿No es más lento? Estudios muestran baja en volumen a corto plazo, pero drástica reducción de errores y riesgo empresarial (compartir conocimiento evita dependencias críticas).

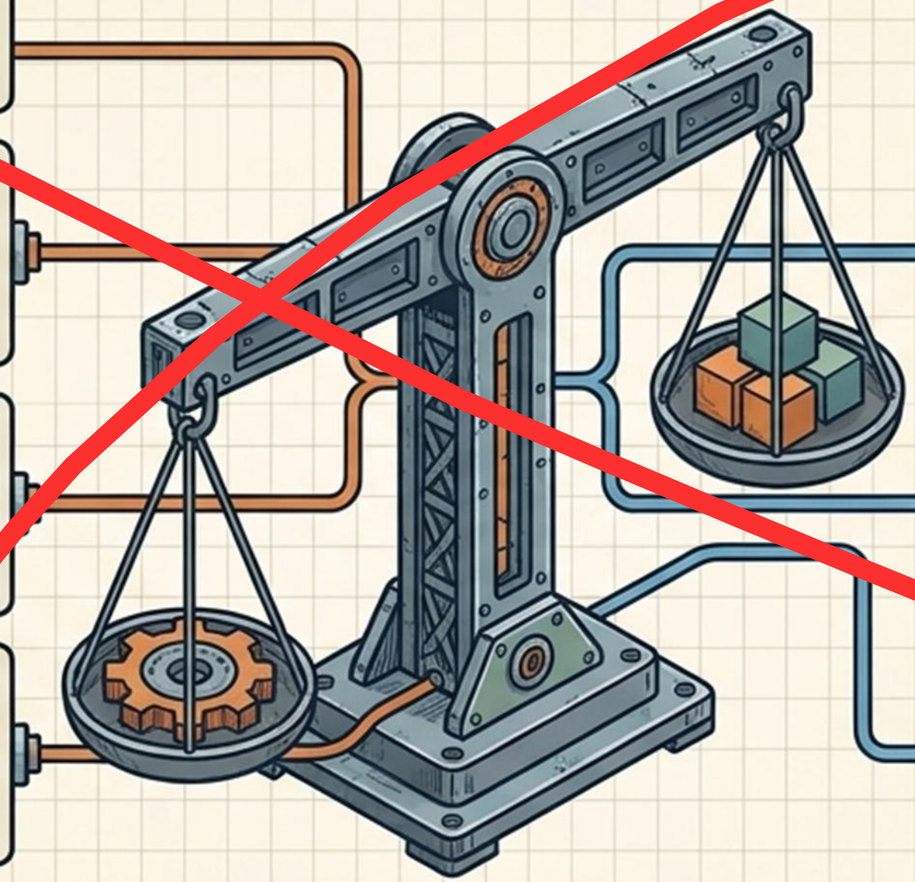
El debate XP: Críticas y Respuestas

Críticas

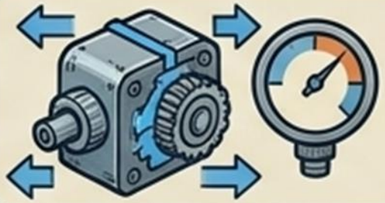
1. **Volatilidad:** Cambios informales descontrolan el alcance.

2. **Requerimientos Informales:** Las tarjetas olvidan detalles críticos.

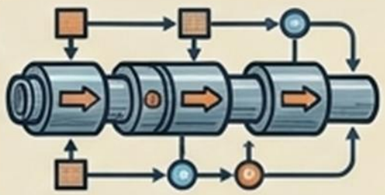
3. **Falta de Diseño Formal:** Sistemas complejos se vuelven frágiles.



Defensas XP



Las iteraciones controlan el impacto del cambio.

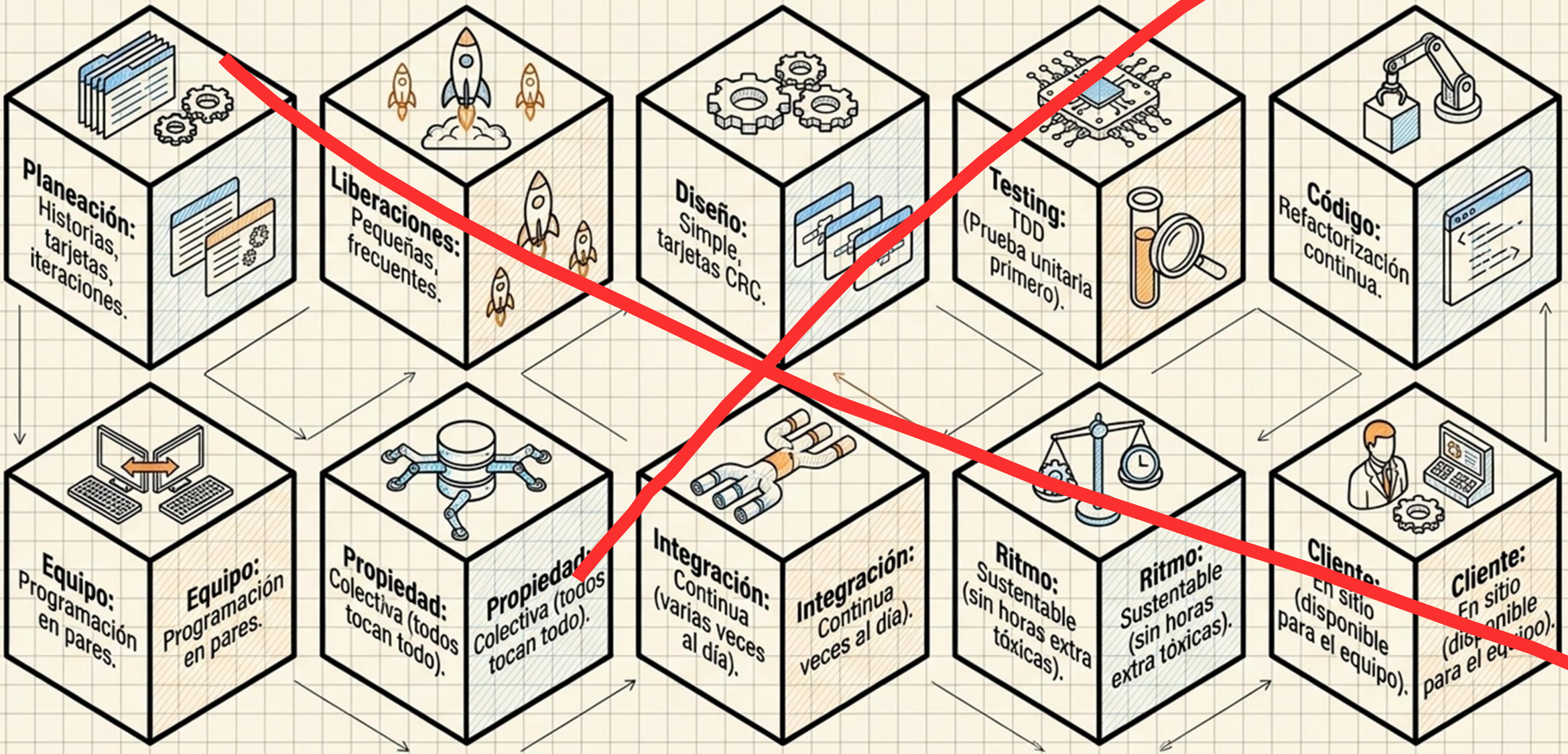


Los documentos formales se vuelven obsoletos igual de rápido.

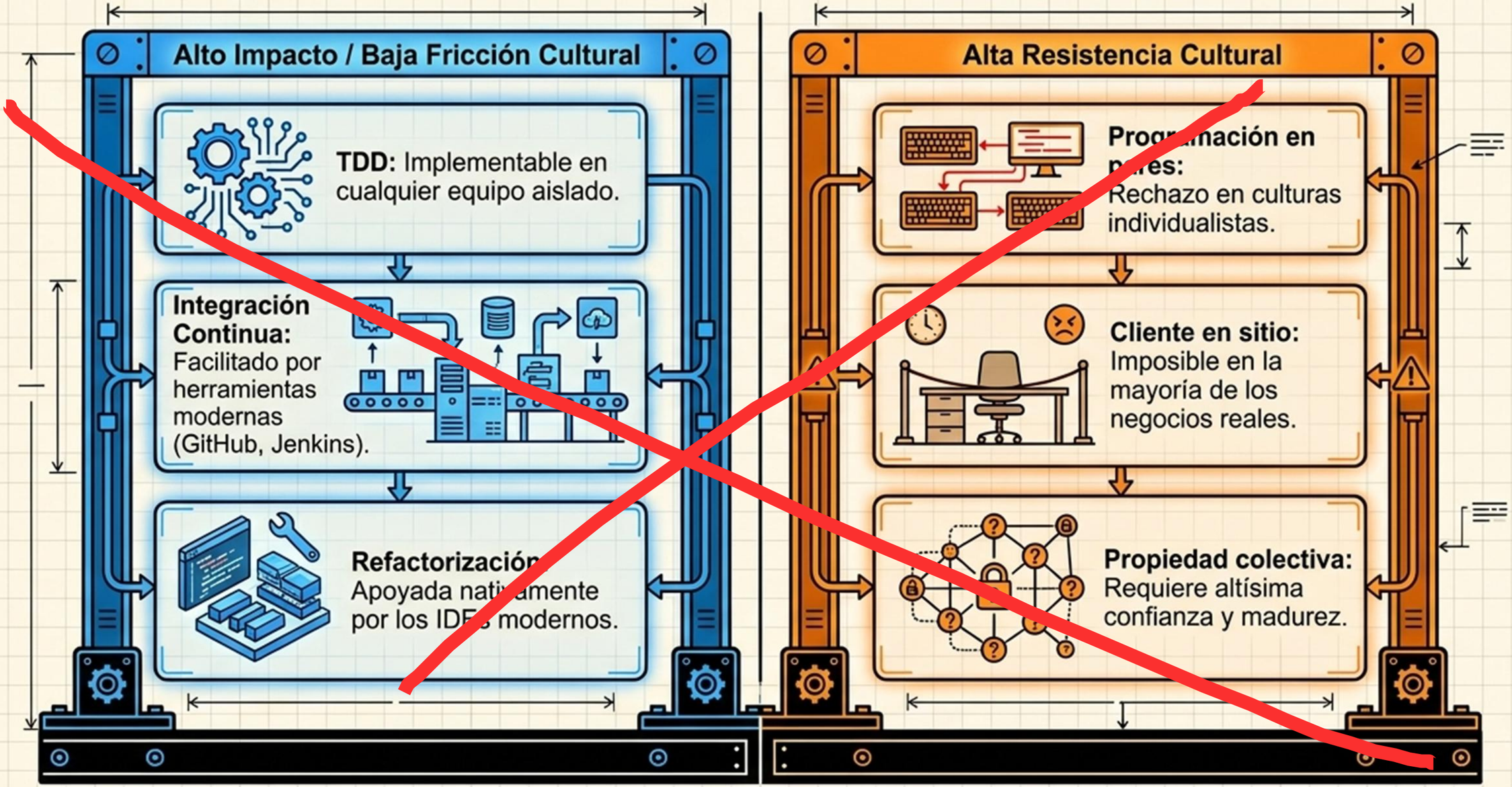
La **simplicidad radical** previene la complejidad innecesaria.

El Veredicto (Pressman): La clave no es ser dogmático, sino identificar debilidades y adaptar XP a la organización.

Las prácticas XP de un vistazo



XP a la carta: Adopción pragmática



Síntesis: La máquina XP ensamblada

Prácticas Técnicas

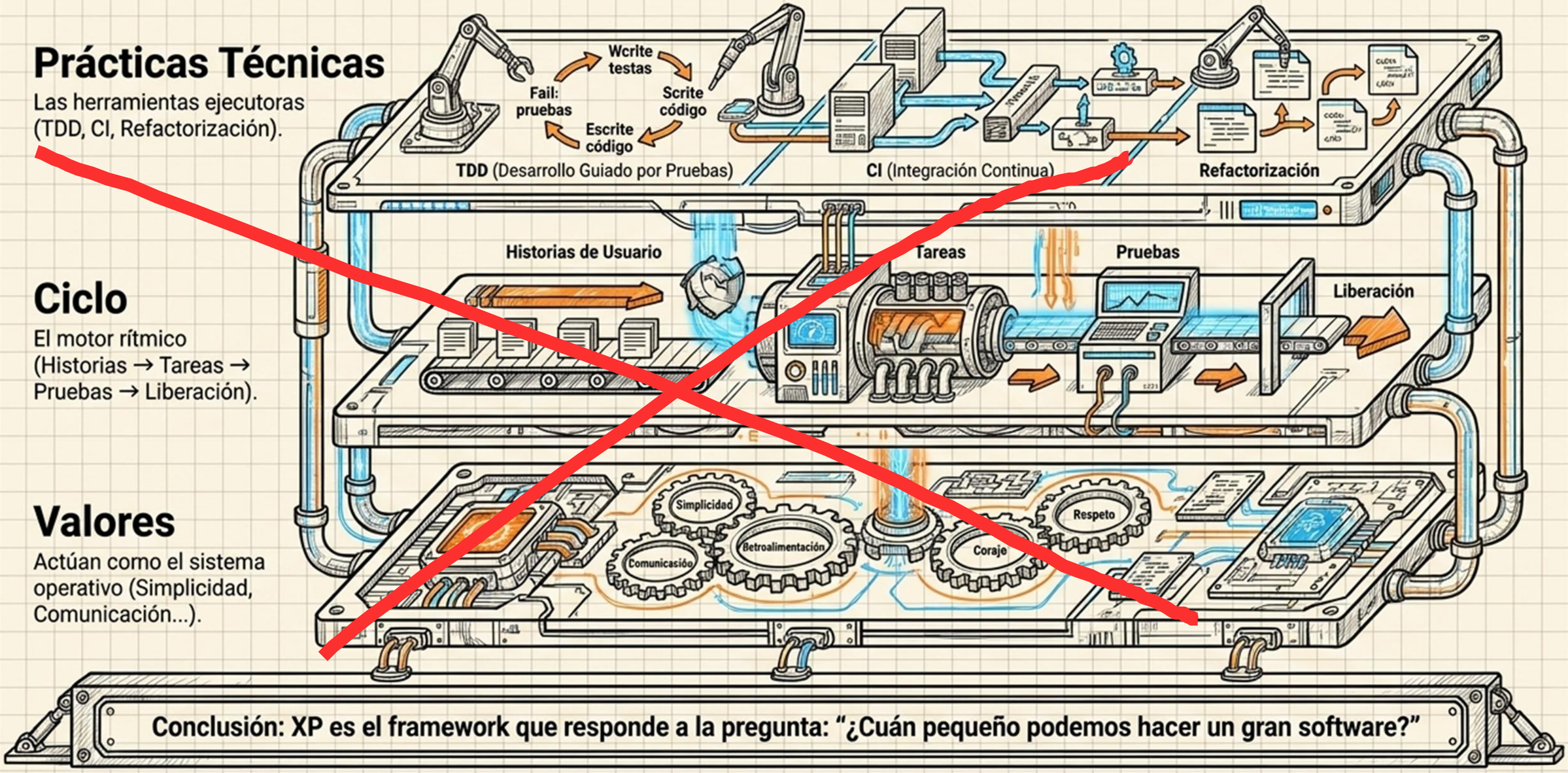
Las herramientas ejecutoras (TDD, CI, Refactorización).

Ciclo

El motor rítmico (Historias → Tareas → Pruebas → Liberación).

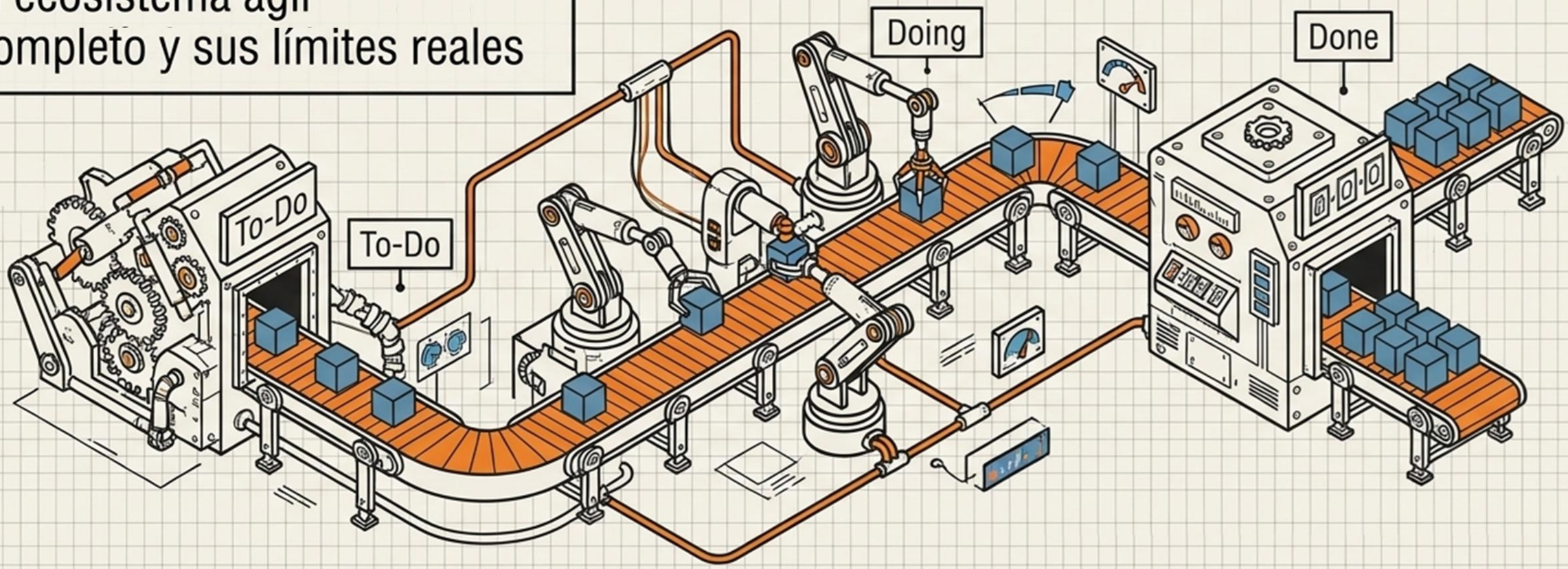
Valores

Actúan como el sistema operativo (Simplicidad, Comunicación...).

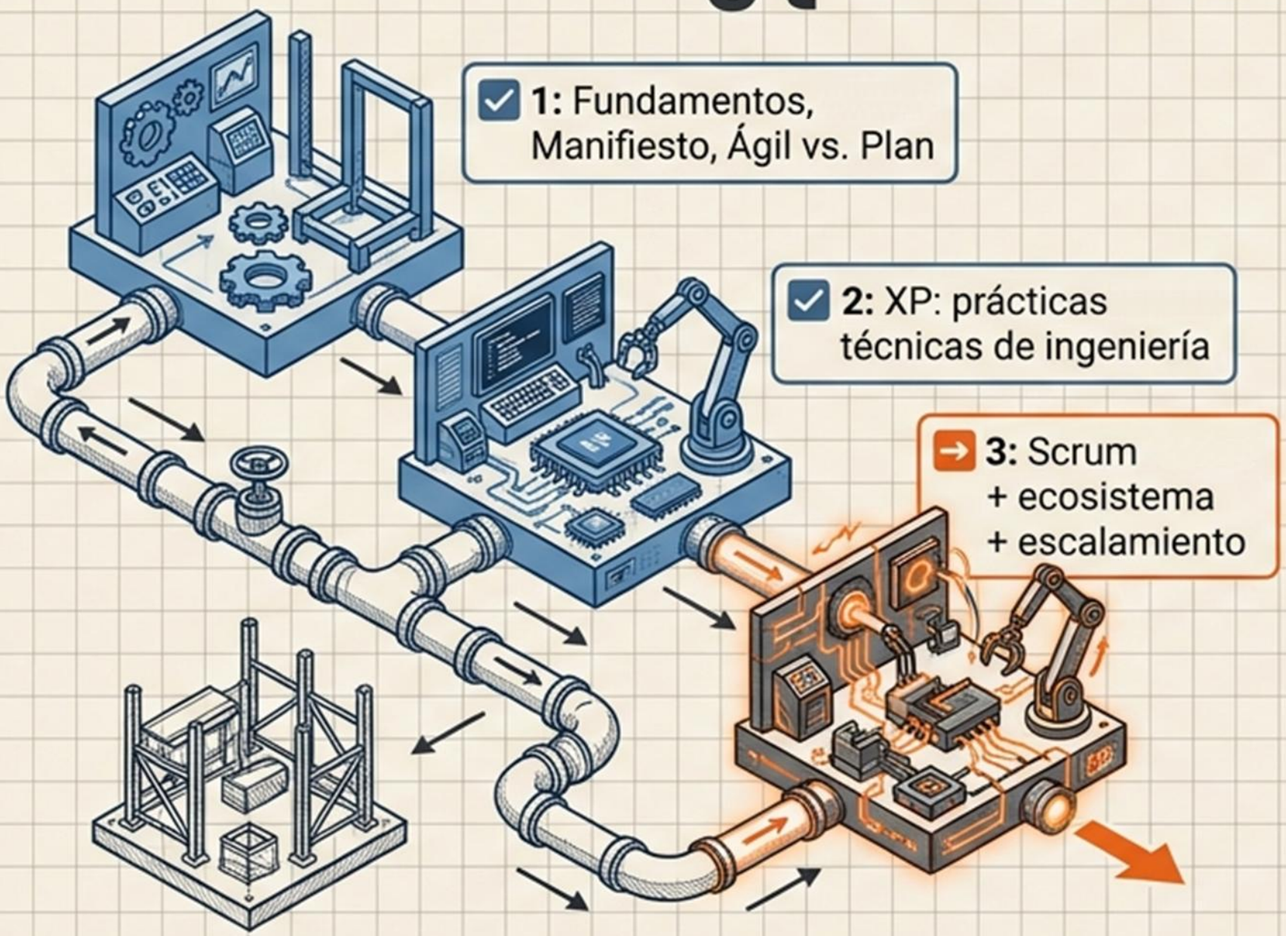


Scrum, otras metodologías y escalamiento

El ecosistema ágil completo y sus límites reales



¿Qué cerramos hoy?



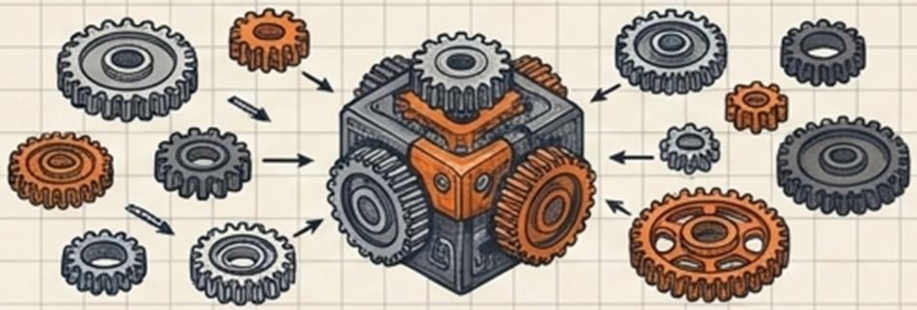
Lo que falta para cerrar el panorama

- **Scrum:** El framework de gestión más usado hoy.
- **El ecosistema:** Otras metodologías clave.
- **El límite real:** ¿Qué pasa en sistemas grandes?
- **Integración:** ¿Cómo usar todo junto?

Key Insight: Esta clase **cierra el ciclo**. Hoy obtienen el panorama completo (gestión + técnica) para tomar decisiones metodológicas en proyectos reales.

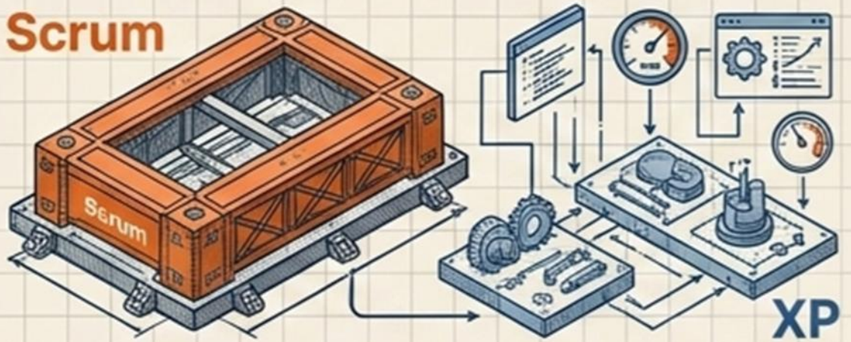
Scrum: Gestión Ágil de Proyectos

El Concepto



- **Origen del rugby**: colaboración intensa, el equipo avanza como un solo bloque.
- **Enfoque**: Administración iterativa en ciclos cortos (Sprints).

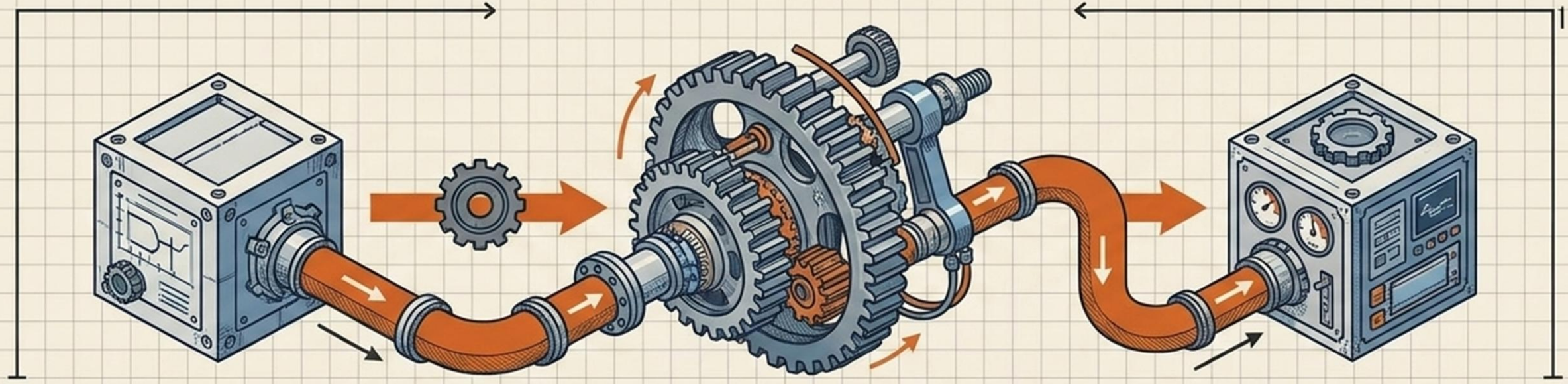
XP vs Scrum



- **Scrum**: NO prescribe cómo escribir código. Dicta cómo gestionar el proyecto.
- **XP**: Dicta cómo construir el software (prácticas técnicas).
- **Conclusión**: Son complementarios, no rivales.

Key Insight: Scrum es hoy el **framework más usado** en la industria de productos. Conocer su mecánica real es un requisito profesional.

Estructura General del Proceso Scrum



Planeación del Bosquejo

- Objetivos generales del proyecto;
- Arquitectura de alto nivel;
- Primera versión del Product Backlog.

Ciclos Sprint (El Corazón)

- Iteraciones de duración fija (2-4 semanas);
- Produce un incremento funcional;
- Repetitivo hasta completar.

Cierre del Proyecto

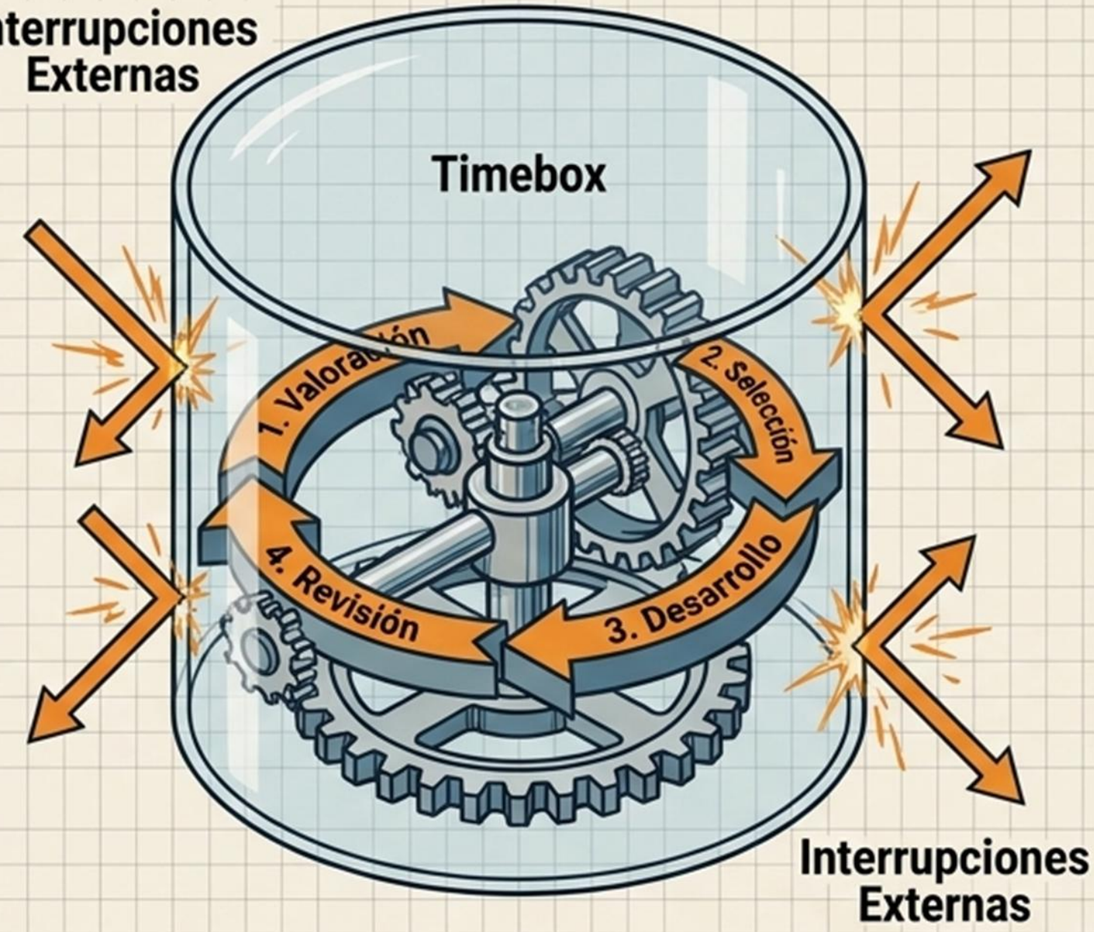
- Documentación completada;
- Lecciones aprendidas;
- Entrega del producto final.

Key Insight: La fase **de bosquejo** reconoce que, incluso en metodologías Ágiles, se requiere una arquitectura inicial antes de encender la maquinaria.

El Sprint: Trabajo Aislado y Sin Interrupciones

El Concepto

- **Unidad de planeación de tiempo fijo** (caja de **tiempo: 2-4** semanas).
- **Regla de Oro:** Fechas fijas. No se aceptan cambios externos **durante el sprint**. Si no logra el objetivo, se reduce el alcance, NO se amplía el tiempo.



Las 4 Etapas Internas

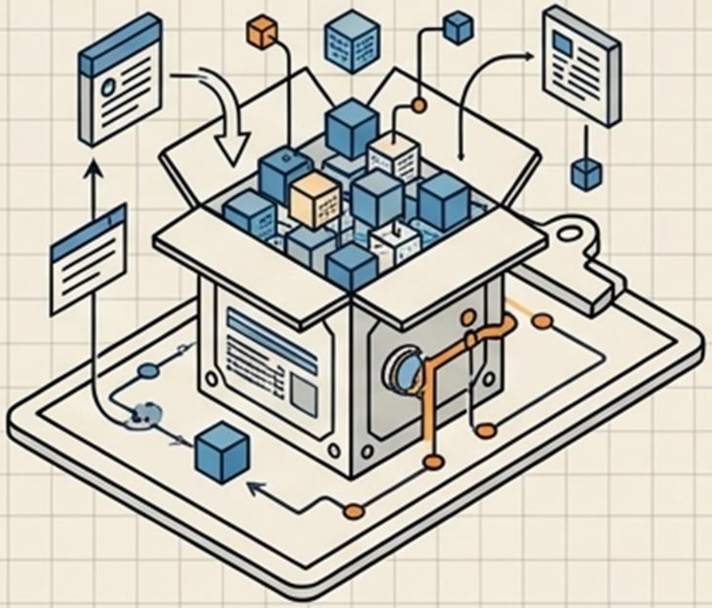
1. **Valoración:** Revisión del backlog y riesgos con cliente.
2. **Selección:** El equipo elige qué desarrollar.
3. **Desarrollo:** Equipo aislado construyendo.
4. **Revisión:** Presentación de funcionalidad y reinicio.

Key Insight: El aislamiento es intencional y vital. Protege al motor de interrupciones destructivas. El Scrum Master actúa como el escudo.

UTN FACULTAD REGIONAL RAFAELA

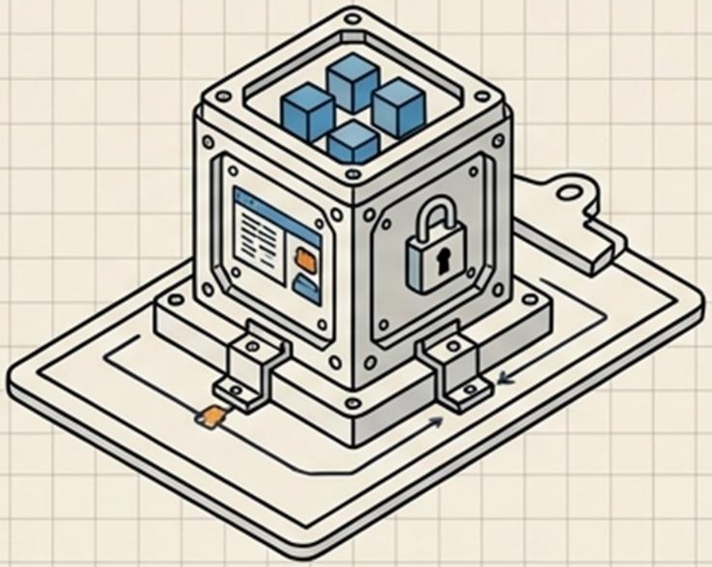
La Materia Prima: Backlog, Sprint Backlog e Incremento

Product Backlog



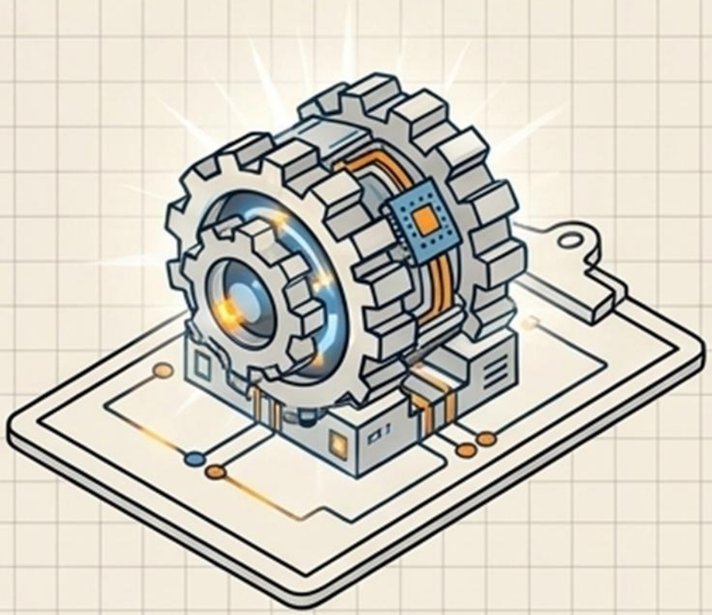
- Lista priorizada de trabajo futuro.
- Fluido: el cliente puede agregar ítems siempre.
- La única fuente de verdad.

Sprint Backlog



- Selección extraída para el sprint actual.
- Estático: Una vez comprometido, NO CAMBIA.
- Es el contrato del equipo.

Incremento

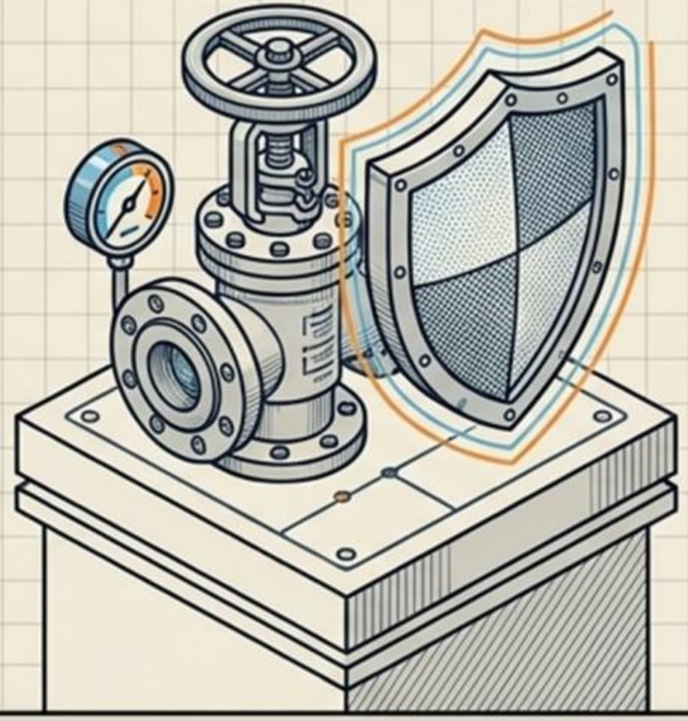


- El resultado tangible del sprint.
- Funcional y potencialmente entregable (código real que opera).

Tres Roles Estructurales (Sin Jerarquía)

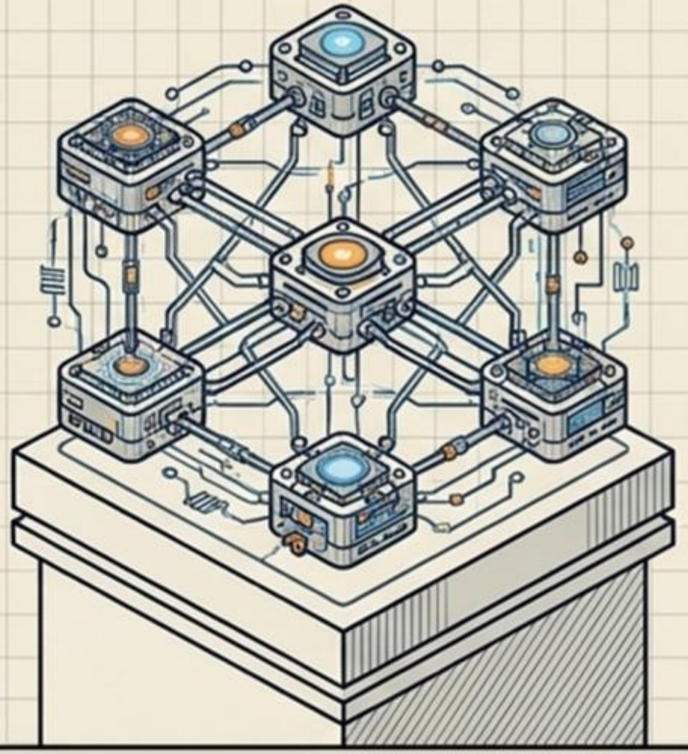
Product Owner

- Representa intereses del negocio.
- Define y prioriza el Product Backlog.
- Es el cliente interno.



Scrum Master

- Facilitador, no es el jefe.
- Rastrea avances, destraba obstáculos.
- Escudo protector del equipo.



Equipo de Desarrollo

- Auto-organizado: Eligen cuánto trabajo tomar.
- Toman decisiones técnicas autónomas.
- Responsabilidad colectiva.

Key Insight: "Nada desmotiva tanto a un equipo como que alguien establezca compromisos por él. Nada motiva más que la responsabilidad propia."

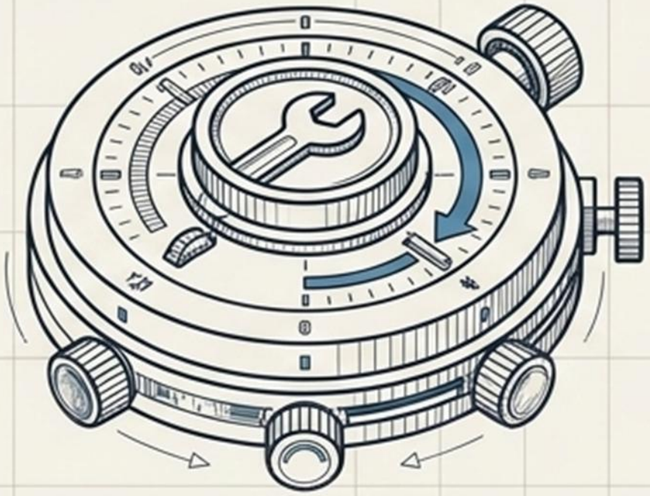
Ceremonias: Comunicación Estructurada, Cero Burocracia

Daily Scrum (15 Minutos)

- Equipo de pie (sincronía rápida).
- Detecta problemas temprano.
- 3 preguntas: 1. ¿Qué hiciste ayer?
  2. ¿Qué obstáculos hay?
  3. ¿Qué planeas hoy?

Sprint Review (Fin de Ciclo)

- Se presenta el Incremento a stakeholders.
- Evaluación de funcionalidad real.
- Retroalimentación directa.



Retrospectiva (Ajuste)

- El equipo reflexiona sobre el proceso, no el producto.
- ¿Qué salió bien? ¿Qué mejorar?
- Mantenimiento y mejora continua.

La Evidencia Empírica: Por qué funciona Scrum

1. Desglose: Sistemas masivos se atacan en piezas manejables.

2. Absorción: Los requerimientos inestables rebotan en el Sprint pero alimentan el Backlog.

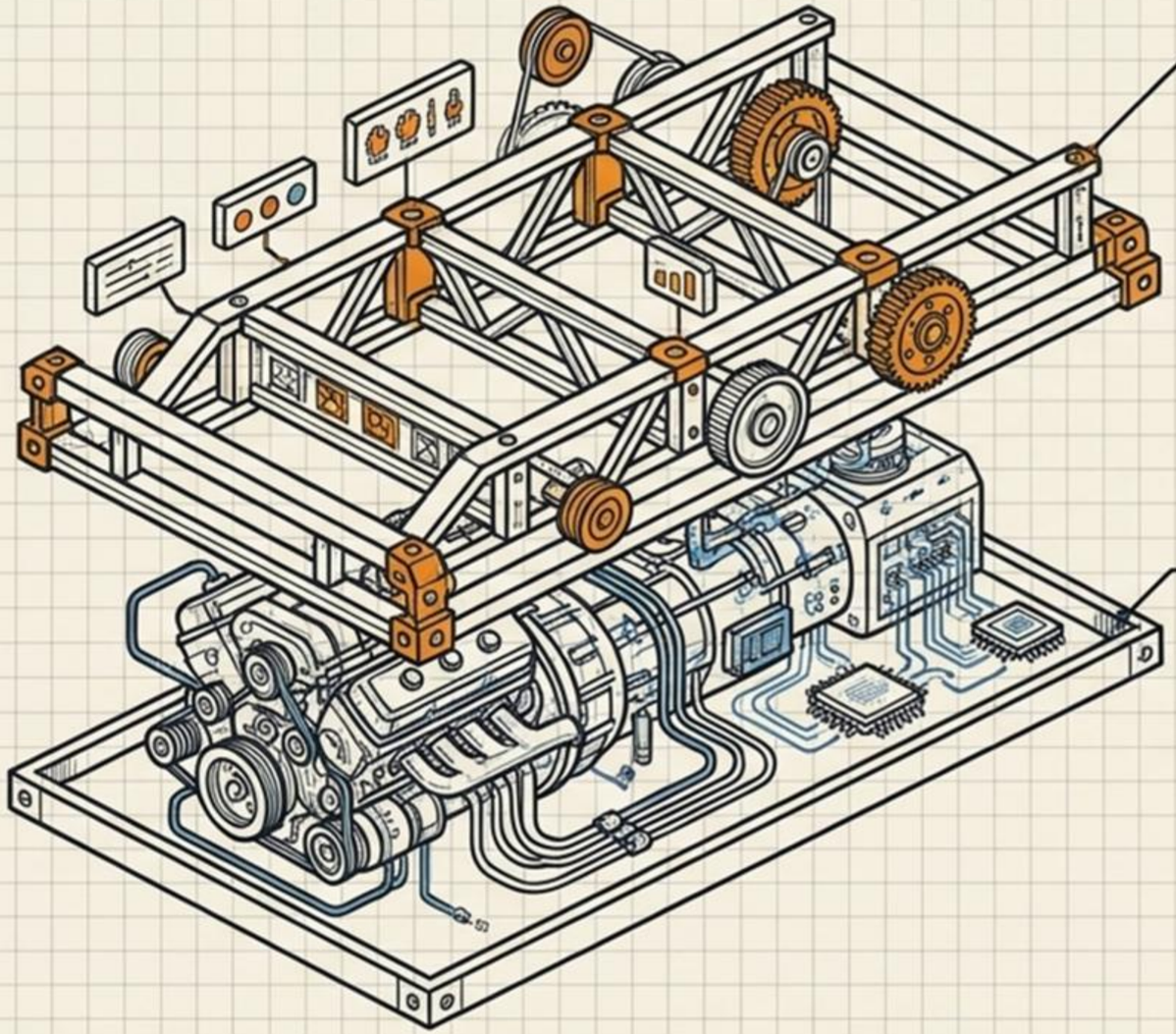
3. Visibilidad: Reuniones diarias destruyen los silos de información.

4. Confianza: Entregas a tiempo construyen moral con el cliente.

5. Corrección: Aprendizaje continuo evita construir software inútil.

Limitación Técnica: Scrum fue diseñado para equipos co-ubicados. En entornos remotos requiere infraestructura colaborativa pesada.

El Dúo Dinámico Industrial: Scrum + XP



Capa Superior (Scrum)

Gestiona prioridades, fechas y ritmo.

Componentes: Backlog → Planning → Sprint → Daily → Review

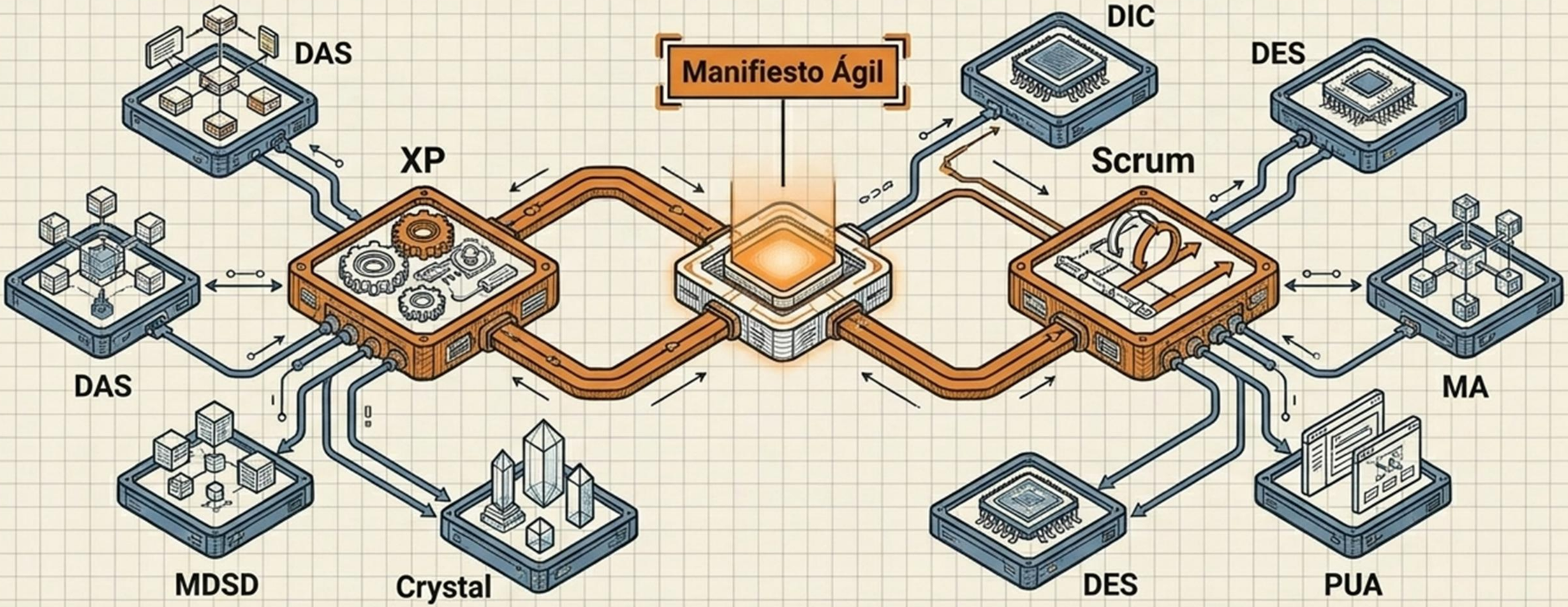
Capa Inferior (XP)

Garantiza la calidad del código y la manufactura fina.

Componentes: TDD, CI/CD, Pares, Refactorización

El Ecosistema Ágil Completo

XP y Scrum dominan la industria, pero el ecosistema posee herramientas altamente especializadas. Todas comparten el código genético del Manifiesto, pero varían su diseño según el terreno.

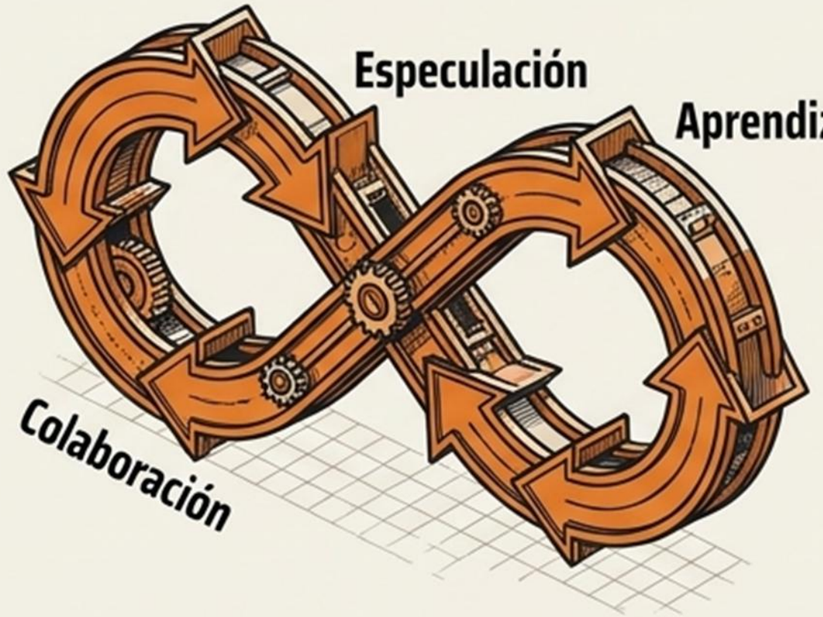


Key Insight: Conocer el ecosistema completo permite usar la herramienta correcta en lugar de forzar un martillo (Scrum) para cada tornillo del proyecto.

UTN FACULTAD REGIONAL RAFAELA

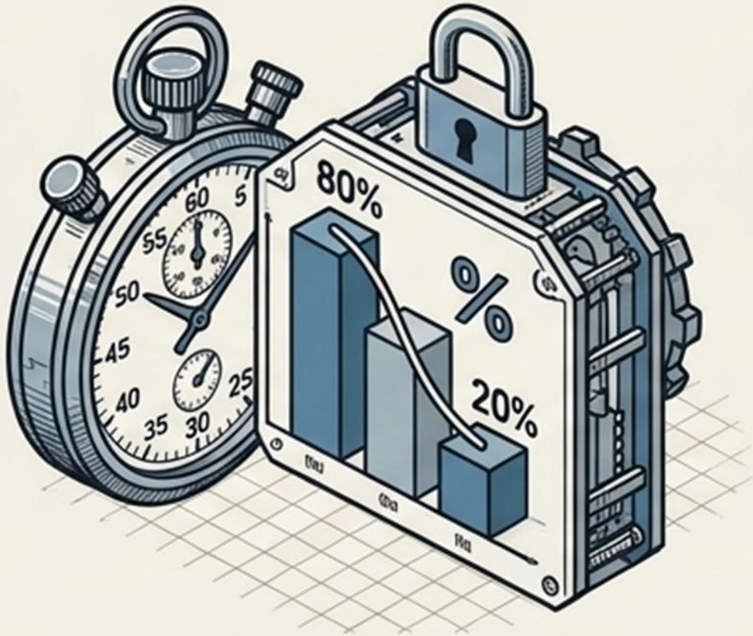
DAS y MDSD: Adaptación y Cajas de Tiempo

DAS (Highsmith)



- **Foco:** Colaboración humana ante incertidumbre extrema.
- **Fases:** Especulación → Colaboración → Aprendizaje.
- **Filosofía:** El plan original fallará; el sistema está diseñado para corregir el rumbo empíricamente.

MDSD (DSDM Consortium)



- **Foco:** El tiempo es inamovible (Timeboxes), el alcance es flexible.
- **Regla Pareto:** 80% del valor se entrega en el 20% del tiempo.
- **Táctica:** Suficiente trabajo para avanzar sin sobre-pulir.

Crystal, DIC y DES: Flexibilidad, Trazabilidad y Lean

Crystal (Cockburn)

- Familia de métodos (Clear, Yellow, Orange) que cambia según tamaño y criticidad del equipo.
- Máxima maniobrabilidad (Juego cooperativo).

DIC / FDD (Coad)

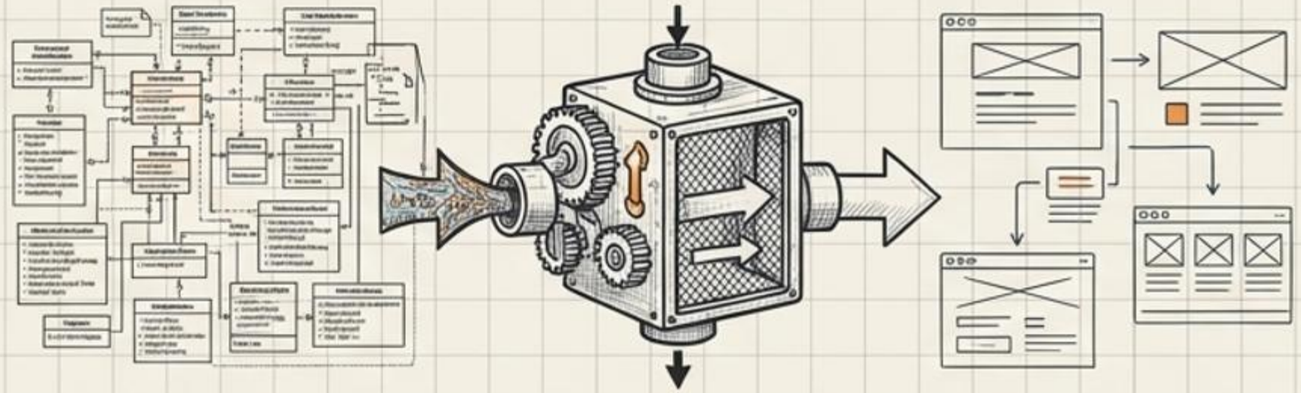
- Desarrollo Impulsado por Características (< 2 semanas).
- Estructura formal, inspecciones rigurosas y excelente trazabilidad.

DES / Lean (Poppendieck)

- Software inspirado en la manufactura de Toyota.
- Eliminar desperdicios (Burocracia, features no usados, bugs).
- Entregar rápido.

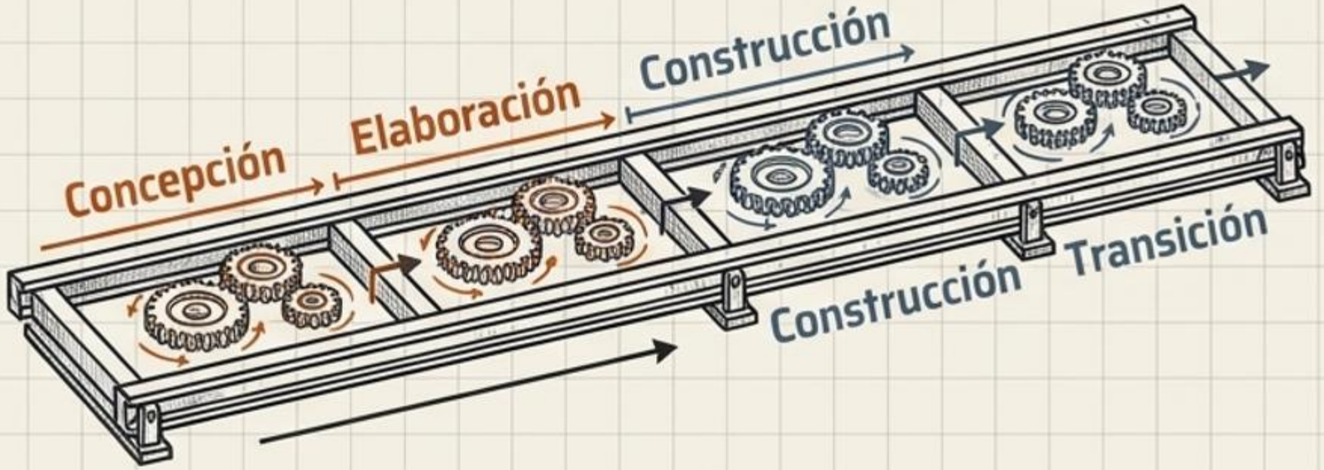
MA y PUA: Diseño sin Asfixia Burocrática

MA - Modelado Ágil (Ambler)



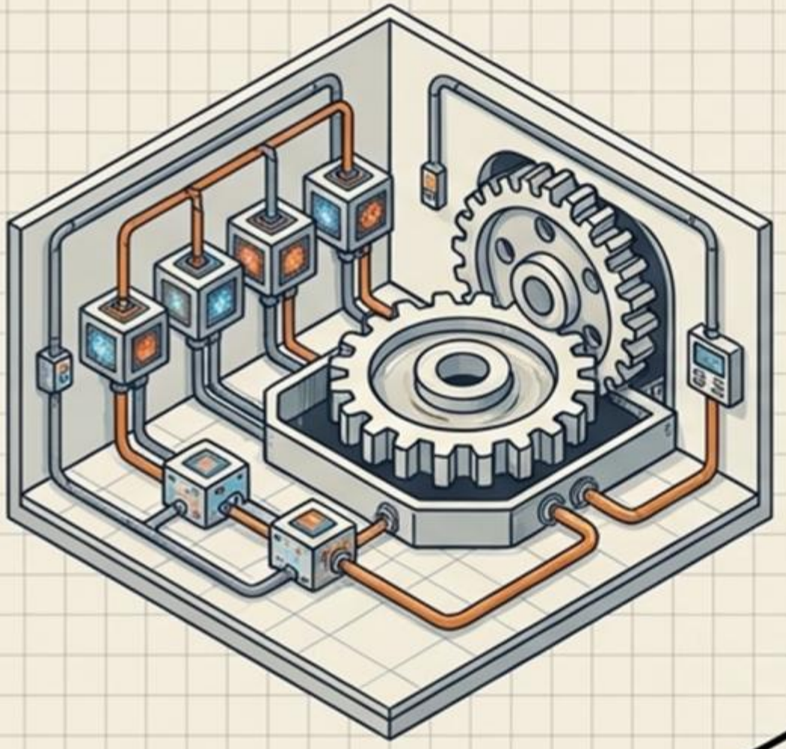
- Principios: Modelar con propósito.
- Contenido > Representación (un esquema feo pero útil supera a un UML perfecto incomprensible).
- Viajar ligero.

PUA - Proceso Unificado Ágil (Ambler)



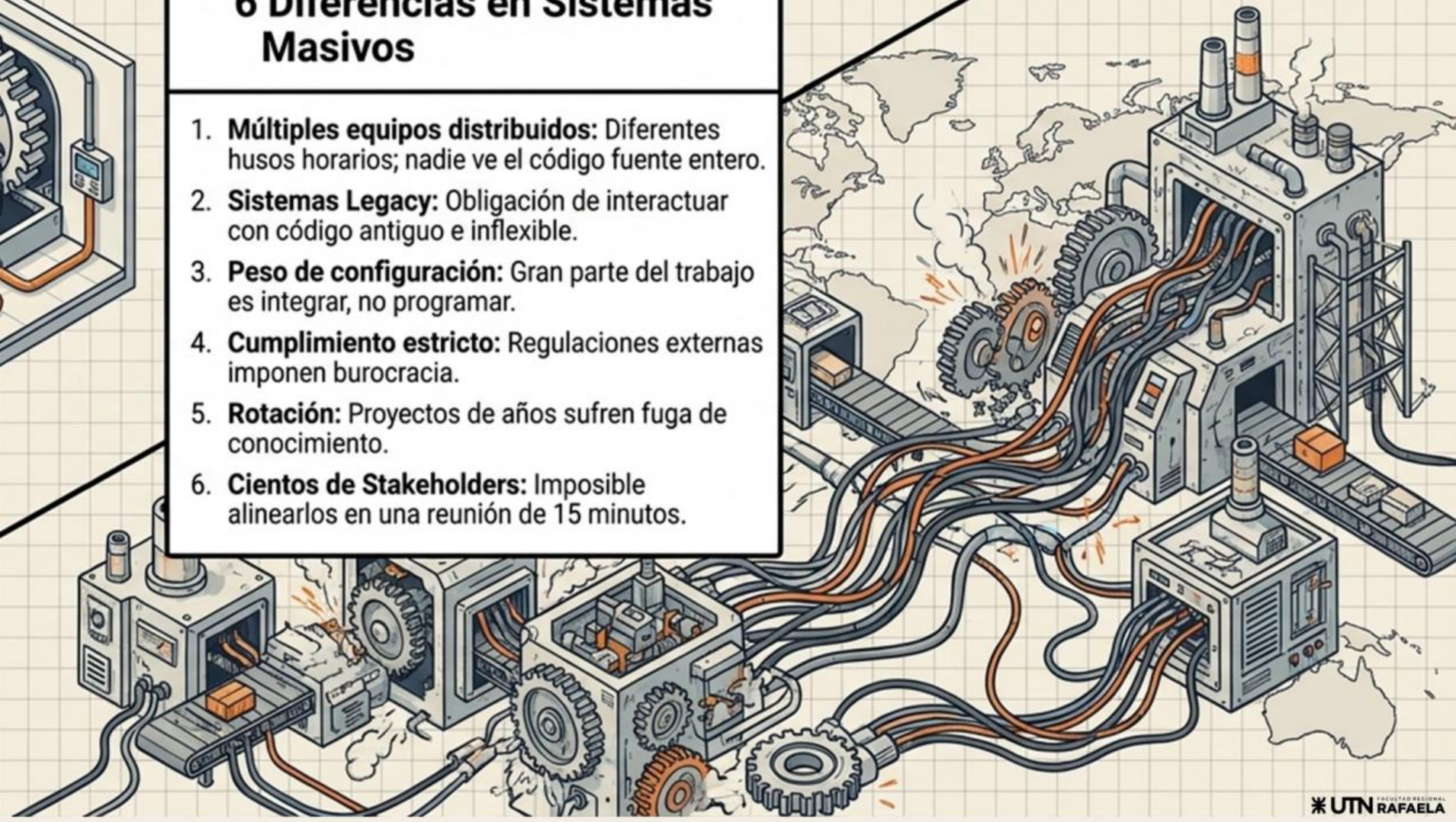
- **Lema:** Serial en lo grande, iterativo en lo pequeño.
- **Retiene fases clásicas** (Concepción, Elaboración) pero ejecuta micro-iteraciones por dentro.
- **El puente perfecto** desde esquemas en cascada.

La Realidad del Escalamiento: Cuando el Sistema Crece

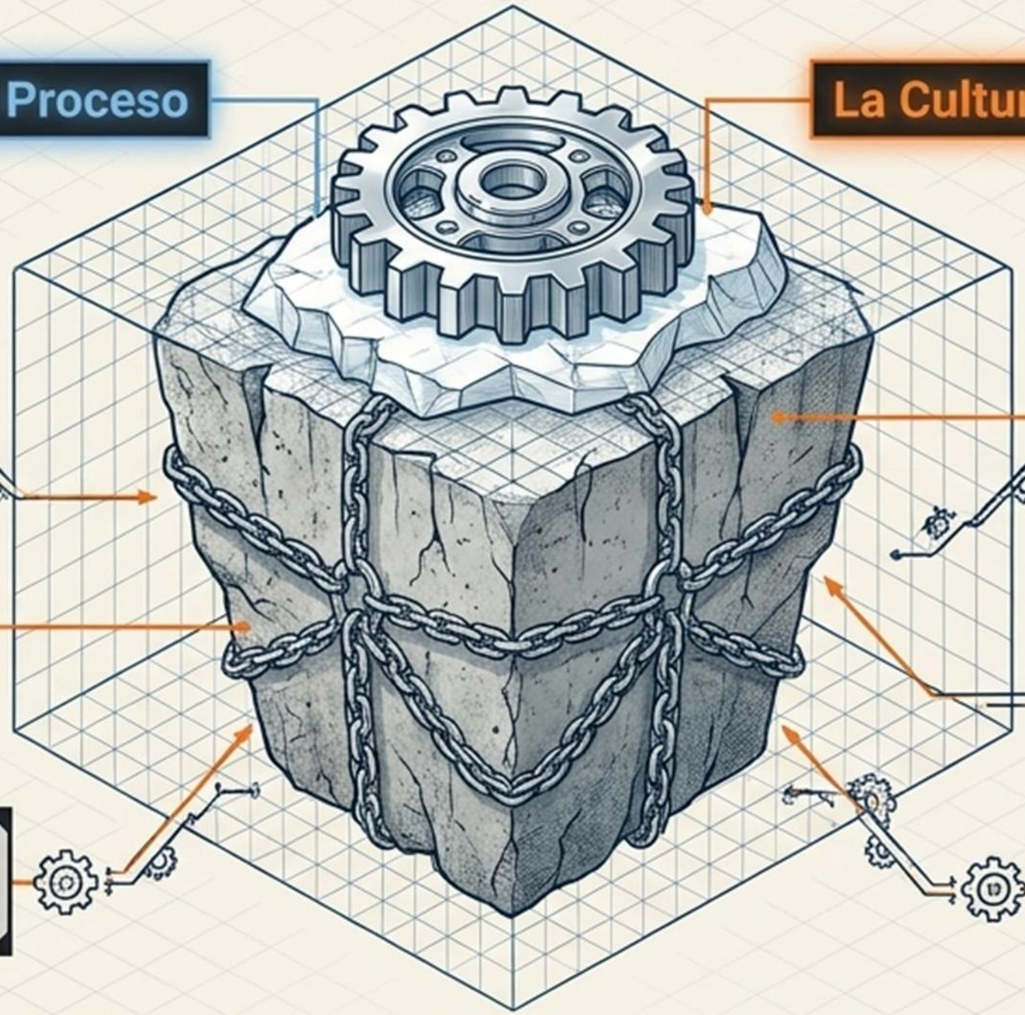


6 Diferencias en Sistemas Masivos

1. **Múltiples equipos distribuidos:** Diferentes husos horarios; nadie ve el código fuente entero.
2. **Sistemas Legacy:** Obligación de interactuar con código antiguo e inflexible.
3. **Peso de configuración:** Gran parte del trabajo es integrar, no programar.
4. **Cumplimiento estricto:** Regulaciones externas imponen burocracia.
5. **Rotación:** Proyectos de años sufren fuga de conocimiento.
6. **Cientos de Stakeholders:** Imposible alinearlos en una reunión de 15 minutos.



El Problema Cultural: El Límite Final



1. Gerencia Inexperta: Miedo corporativo a alcances que fluctúan.

2. Calidad Incompatible: El ISO 9001 choca frontalmente con los post-its.

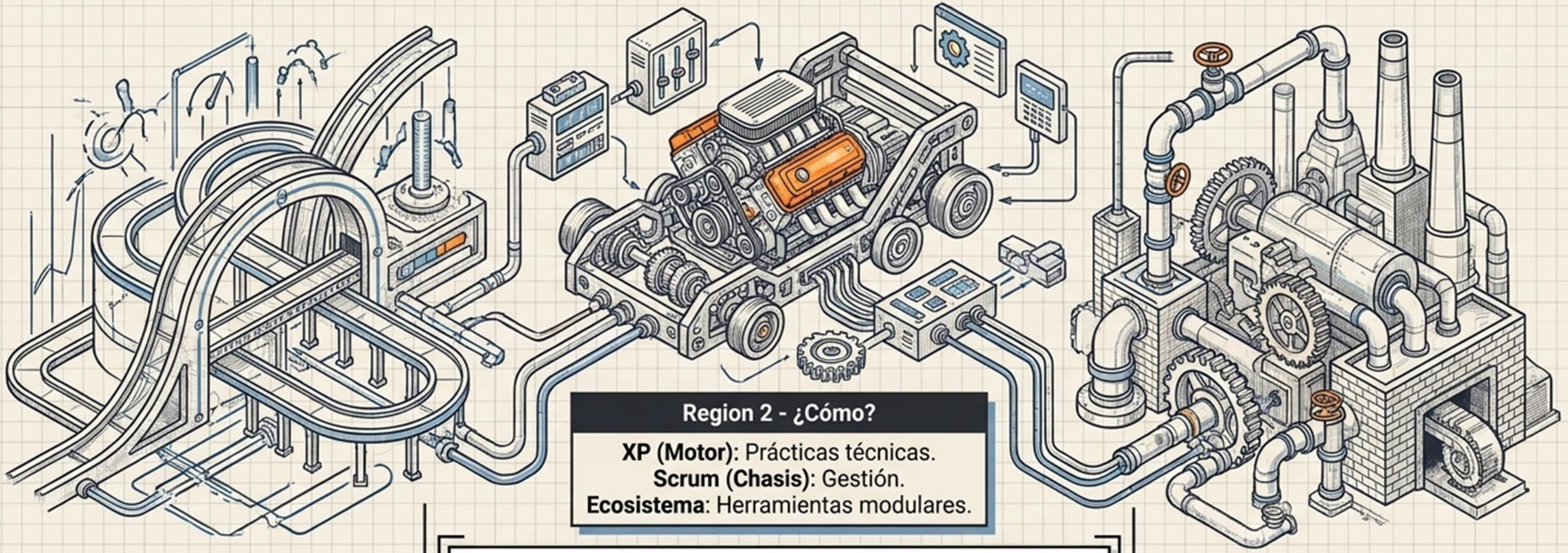
3. Heterogeneidad de Talento: Ágil exige alta autonomía; las macro-empresas tienen niveles de talento muy dispares.

4. Resistencia Institucional: Siempre documentamos así por 20 años.

5. Aprobaciones en Cascada: Requerir firmas físicas para refactorizar destruye el motor Ágil.

Key Insight: Introducir Ágil a gran escala NO es un cambio de software. Es una intervención masiva de psicología organizacional.

Síntesis Maestra: La Arquitectura del Desarrollo Ágil



Region 1 - ¿Por qué?

Manifiesto Ágil (4 Valores).
Entornos inestables.

Region 2 - ¿Cómo?

XP (Motor): Prácticas técnicas.
Scrum (Chasis): Gestión.
Ecosistema: Herramientas modulares.

Region 3 - Límites?

Escalamiento distribuido, choque cultural, integración legacy.

La pregunta profesional madura no es ¿Ágil o Tradicional?, sino: ¿Qué engranajes ágiles específicos resuelven el problema actual de mi entorno, equipo y restricciones?